



**Trinity
College
Dublin**

The University of Dublin

QUBO and Ising Formulations of Optimisation Problems

Simulated annealing from Combinatorial Optimisation to Optimal
Control

Cian Gregg

(draft one of the write up)

A summer research project

Supervised by Professor Stefano Sanvito and Michael Mitchell

Contents

1	Introduction	3
2	Monte Carlo Markov Chain methods and Ising Models	4
2.1	Monte Carlo Methods	5
2.1.1	Calculating π with Monte Carlo Methods	5
2.1.2	Monte Carlo Sampling	6
2.1.3	Markov chains	6
2.2	Monte Carlo Markov Chain Algorithms	7
2.2.1	Metropolis-Hastings Acceptance Criterion	8
2.2.2	Metropolis algorithm	9
2.3	The Ising Model	9
2.3.1	Implementing metropolis for a 2d lattice	10
2.3.2	Observables	11
2.4	Simulated Annealing	14
2.4.1	Implementing annealing	14
2.4.2	Generalising: Spin Glass Theory	15
2.4.3	Optimisation: JIT Compilation with Numba	15
2.4.4	Spin to binary: QUBO	16
2.5	Improving the annealer	17
2.5.1	sweeping until converged	17
2.5.2	More effective starting and ending temperatures	18
2.5.3	Comparison: dwave	18
3	Constructing Energy functions	19
3.1	The General Structure	19
3.2	Number Partitioning	20
3.2.1	Implementing Number Partitioning	22
3.3	Graph Partitioning	23
3.3.1	Implementation Graph Partitioning	25
3.4	Graph Colouring	27
3.4.1	Implementing Graph Colouring	28
3.5	Hamiltonian Cycle/Path	29
3.5.1	Implementing Hamiltonian Cycle	30
3.6	Travelling Salesman	31
3.6.1	Implementing TSP	32
3.6.2	Distance Based Weighting	33
3.7	Concluding the Combinatorial Problems	36
4	QUBO Formulation of Rocket Optimal Control	36
4.1	Encoding Continuous Variables	36
4.1.1	Integer Encoding	36
4.1.2	Decimal place expansion	37
4.1.3	Weighted Encoding (four bits per variable)	37
4.2	Rocket Take-off Problem Formulation	37
4.2.1	Physical Model	38
4.2.2	Time Discretisation	38
4.2.3	Optimisation Problem	38

4.3	Constructing the Energy Function	39
4.3.1	Fuel Objective	39
4.3.2	Starting and Ending Constraints	39
4.3.3	Dynamics Constraints	39
4.3.4	Complete Energy function	40
4.4	Building the QUBO Matrix	40
4.4.1	General Quadratic expansion	40
4.4.2	Fuel Contribution	41
4.4.3	continuous encoding	41
4.4.4	Terminal and starting contributions	43
4.4.5	Dynamic Constrains	43
4.5	Final Q Matrix	45
4.6	Implementation	45
4.6.1	Indexing and Encoding	45
4.6.2	The General Expansion	46
4.6.3	Building Each Energy Term	46
4.6.4	Assembling Q	48
4.6.5	Decoding a solution	48
4.7	Annealing and Results	49
4.7.1	Not finding the Minima	49
4.8	Tuning the constraint coefficients	49
4.8.1	Residuals	50

5	Conclusion	52
----------	-------------------	-----------

Abstract

We explored reformulating optimisation problems as the search for minimum energy states of an Ising or QUBO energy landscape. We developed a Monte Carlo Markov Chain framework to find these low energy states, building up from the Metropolis algorithm to simulated annealing. With this annealing framework in place, we then investigated a sequence of established combinatorial problems, working through their constructions in detail. This taught us the techniques needed to translate constraints and objectives into energy contributions. We then extended this to a continuous problem, *a mixed-integer optimal control formulation of a 1D rocket trajectory*, encoding continuous position & velocity variables as binary and enforcing the rocket's dynamics as constraint terms. Annealing this resulting Q matrix showed that optimal control problems of this kind can be expressed in a form that naturally extends to probabilistic computing hardware.

1 Introduction

Probabilistic computing offers an alternative approach to conventional computing by introducing stochastic behaviour into computation. Networks of probabilistic bits (p-bits) fluctuate between 0 or 1, with their collective behaviour described by a Boltzmann distribution. This provides a physical representation of Ising machines that can be applied to solving optimisation problems.

Throughout this project, our aim is to re-express optimisation problems as the search for the minimum of an energy landscape. For each problem we construct an energy function $E(X)$ such that the low energy states correspond to valid solutions, with the lowest corresponding to an optimal valid solution. We can picture this energy function as a landscape of hills and troughs, containing many local minima, and a global minimum we're looking for.

The connection between this energy landscape and probabilistic computing, comes from the Boltzmann distribution,

$$P(s) = \frac{e^{-\beta E(x)}}{Z}, \quad Z = \sum_j e^{-\beta E(x_j)},$$

where Z is the partition function and $\beta = 1/k_b T$ is our inverse temperature. This distribution assigns higher probability to lower energy configurations and lower probabilities to higher energy states. A system sampling from this distribution will preferentially occupy the lower energy states.

Our computational goal is therefore to efficiently search this energy landscape for its global minimum. Directly sampling from the Boltzmann distribution is generally intractable, since calculating the partition function Z requires a sum over all possible states. Instead, we use Monte Carlo Markov Chain methods to sample from the distribution indirectly. In particular, the Metropolis algorithm, which allows the system to explore the energy landscape by accepting some higher energy swaps probabilistically, preventing the system getting stuck in the first local minima. This can be pictured as placing a ball at a random point on the landscape and letting it roll downhill, allowing the ball to probabilistically roll uphill sometimes. Simulated annealing builds on idea by beginning the search at high temperatures, allowing broad exploration of the landscape, and slowly cooling the system narrowing the search to the true minimum. Throughout this project we build a computational framework for implementing these ideas for both Ising and QUBO formulated energy functions.

To construct these energy function, we first want to look at an Ising model. An Ising system represents variables as lattice of spins $s_i \in \{-1, +1\}$, with interactions between pairs of spins described by J_{ij} and and external bias described by h_i . The energy of this system given by,

$$H = - \sum_{\langle ij \rangle} J_{ij} s_i s_j - \sum_i h_i s_i$$

Finding the ground state of the system is itself an energy minimisation problem, providing a natural representation for probabilistic computing in which the stochastic system explores the landscape.

For optimisation problems, its convenient to represent variables as binary $x_i \in \{0, 1\}$ rather than spins. We can therefore generalise the Ising representation to the Quadratic Unconstrained Binary Optimisation (QUBO) formulation, where the optimisation problem is expressed by a quadratic function of binary variables and a coupling matrix Q ,

$$E(x) = x^T Q x$$

The Ising and QUBO formulations are closely related, providing two representations of the same energy minimisation problem. Within both formulations, constraints are constructed though penalty terms, assigning higher energy values to states that violate the given constraints. The objective function then rewards whatever the goal of optimisation problem was. This process becomes translating constraints and objectives term by term into energy contributions.

With this framework established, the remaining challenge is to construct a suitable energy functions for a given optimisation problem. We build this process working through a sequence of established combinatorial optimisation problems, beginning with simple constraints and progressively introducing more complicated structures including multiple constraints & sequential ordering. This provides a progression through which we can build up the logic to construct our own energy function in the last section.

This leads to the final problem we considered in this project of a *1D rocket trajectory optimal control*. Unlike the previous combinatorial examples, the rocket problem contains both binary decision variables, applying thrust, and continuous physical state variables, position and velocity. To apply the same QUBO annealing framework, these continuous states must first be encoded using binary variables through a decimal place expansion. The rockets dynamics can be incorporated as constraint terms and the control objective, *minimising fuel*, can be built into one energy function. Finally we can anneal this energy function and search for the global minimum using the same MCMC annealing methodology we've built up across this project.

2 Monte Carlo Markov Chain methods and Ising Models

The goal here is to build up a computational framework for minimising arbitrary energy cost functions using Monte Carlo Markov Chain methods, starting from the Ising model as our canonical example.

2.1 Monte Carlo Methods

Lets build up to the main algorithm we need the Metropolis Algorithm, first motivating it with simpler Monte Carlo methods for example calculating π .

2.1.1 Calculating π with Monte Carlo Methods

One of the first task in our computational labs was calculating π using Monte Carlo methods. It follows from the idea that in a unit square has trivially area $A_{\text{square}} = 1$ and the area of a unit circle,

$$A = \pi r^2 = \pi.$$

So if we overlay our unit circle and square each quadrant has an area

$$A_{\text{quadrant}} = \frac{\pi}{4}.$$

and importantly their ratio becomes,

$$\frac{\pi}{4}.$$

Now we can apply our Monte Carlo method as follows:

1. Draw a square and a quadrant of circle
2. Randomly scatter points across our square
3. Calculate the ratio of point that land inside the quadrant to the whole square
4. $4 \times \text{ratio}$: $\frac{\text{points inside}}{\text{all the points}} \approx \pi$

We can run this as calculate pie up to a few decimal places but the more important takeaway is that this was a simple example only because all the points were uniformly distributed across the square. We can pick samples for (x, y) with

```
x = np.random.rand(times)
y = np.random.rand(times)
```

All we are estimating is the areas of shapes by random samples. However in a physical system particles aren't uniformly distributed, rather distributed by the Boltzmann distribution shifting us from calculating areas to calculating thermal averages.

2.1.2 Monte Carlo Sampling

We can formalise this idea of Monte Carlo sampling. Take some function $f(x)$ and get its expectation value, this is the integral of the function weighted by a probability ¹.

Then,

$$\begin{aligned} s(x) &= \int_{\mathbb{X}} p(x) f(x) dx \\ &= \underbrace{\langle f(x) \rangle_{p(x)}}_{\text{statistical physics notation}} \equiv \underbrace{\mathbb{E}_p[f(x)]}_{\text{statistician notation}} \end{aligned}$$

Then our Monte Carlo estimation is,

$$\sigma_n(f(x)) = \frac{1}{n} \sum_i^n f(x^{(i)}) \quad \text{where, } x^{(i)} \text{ are random samples from } p(x)$$

For large enough N the *Law of Large Numbers* tells us that this should converge to our true answer.

In our example above we had $p(x) = \text{Uniform}(0, 1)$ so each sample was easy to generate just random points along $0 \rightarrow 1$. An example like,

$$\{x^{(1)} = 0.12, x^{(2)} = 0.86, \dots x^{(n)} = 0.56\} \in p(x) = \text{Uniform}(0, 1)$$

for large enough n we should see $\sigma_n(f(x)) \approx \int f(x) dx$. Which we saw above with our calculation of pi (up to one decimal place away after only $n = 1000$ samples).

Systems like $p(x) = \text{Uniform}(0, 1)$ are easy to direct sample, however in physical systems we see more complicated distributions like the Boltzmann distribution

$$P(x) \propto e^{-\beta E(x)}$$

Take our canonical example the Ising model with spins σ_i ,

$$P(\sigma_i) = \frac{e^{-\beta H(\sigma_i)}}{Z}$$

direct sampling $P(\sigma_i)$ is practically computationally impossible as we need to calculate Z the partition function a sum over all possible states, for an N lattice that is a sum over 2^N states. To get around this we introduce, the other half of the name, Markov Chains.

2.1.3 Markov chains

Markov chains let us swap our sampling from a complex distribution $p(x)$ with a stochastic process whose long-term behaviour reproduces the target distribution.

¹It could just be a *weighting* but for simplicity I'm taking $p(x) \geq 0$ and it becomes a probability distribution

Consider a sequence of random variables $\{X_1, X_2, \dots, X_n\}$ forming a *stochastic process*. Generally for a stochastic process, we would get a hierarchy of probabilities, forming joint probabilities distributions

$$\begin{aligned} &P_1(X_1 = x_1) \\ &P_2(X_1 = x_1, X_2 = x_2) \\ &\vdots \\ &P_n(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) \\ &\vdots \end{aligned}$$

Where the function $P_n(x_1, x_2, \dots, x_n)$ is the joint probability distribution that X_i takes on the value of x_i .

However the Markov property tells us that the future state depends only on the current state,

$$P(X_{n+1} = x \mid X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) = P(X_{n+1} = x \mid X_n = x_n)$$

This implies that the probability of the entire problem factorises into pairwise transitions probabilities

$$P_n(x_1, \dots, x_n) = P_1(x_1) \prod_{i=1}^{n-1} P(x_{i+1} | x_i)$$

We can use these properties to construct a Markov Chain Monte Carlo algorithm.

2.2 Monte Carlo Markov Chain Algorithms

With the machinery of Markov Chains, we can now write the general shape of an MCMC algorithm. We've left this generic at the moment, not depending on the Ising model or metropolis Hastings acceptance rule yet

1. Pick an initial state $\sigma^{(0)} = \{\uparrow_1, \uparrow_2, \uparrow_3, \uparrow_4, \dots, \uparrow_n\}$ or some other random assignment.
2. Propose a new state σ' , typically flipping one spin $\sigma_i^{(0)} = \uparrow \quad \sigma'_i = \downarrow$
3. Accept or reject the proposed change σ' according our acceptance rule to get closer to equilibrium
4. Update $\begin{cases} \sigma^{(1)} = \sigma' & \text{if accepted} \\ \sigma^{(1)} = \sigma^{(0)} & \text{if rejected} \end{cases}$
5. Repeat steps 2-4 until the system equilibrates
6. Discard the initial *burn in period* to remove any dependence on $\sigma^{(0)}$
7. Use the remaining states as samples from the equilibrium distribution to compute our observables

Steps 2 and 3, how we propose new states and how we decide to accept them will be determined by the choice of acceptance rule.

2.2.1 Metropolis-Hastings Acceptance Criterion

The generic algorithm above leaves the acceptance rate $A(\sigma|\sigma')$ undefined. In this section we'll derive a specific choice of A that ensures that the Markov chain will converge to the target distribution $\pi(\sigma) = P(\sigma) = \frac{e^{-\beta H(\sigma)}}{Z}$.

This target distribution needs to be *stationary* once we've reached equilibrium. This is the idea of *detailed balance* from thermodynamics, in equilibrium the energy flux in both directions should equal. We can define this mathematically as,

$$\pi(\sigma)P(\sigma'|\sigma) = \pi(\sigma')P(\sigma|\sigma')$$

Splitting the transition probability $P(\sigma|\sigma')$ into a proposal distribution $g(\sigma'|\sigma)$, the probability of *proposing* σ' given σ , and an acceptance rate $A(\sigma'|\sigma)$ the probability of *accepting* σ' , gives us

$$\pi(\sigma)g(\sigma'|\sigma)A(\sigma'|\sigma) = \pi(\sigma')g(\sigma|\sigma')A(\sigma|\sigma')$$

Detailed balance holds if we define,

$$A(\sigma'|\sigma) = \min \left(1, \frac{\pi(\sigma')g(\sigma|\sigma')}{\pi(\sigma)g(\sigma'|\sigma)} \right)$$

Now inserting the Boltzmann distribution for π

$$\begin{aligned} A(\sigma'|\sigma) &= \min \left(1, e^{-\beta(E(\sigma')-E(\sigma))} \frac{g(\sigma|\sigma')}{g(\sigma'|\sigma)} \right) \\ &= \min \left(1, e^{-\beta\Delta E} \underbrace{\frac{g(\sigma|\sigma')}{g(\sigma'|\sigma)}}_{=1} \right) \end{aligned}$$

The proposal ratio cancels out as our proposal mechanism is symmetric in nature. We pick one of N spins uniformly at random and flip it so,

$$g(\sigma|\sigma') = g(\sigma'|\sigma) = \frac{1}{N}$$

regardless of which spin we chose.

This leave us with our metropolis algorithms acceptance rate ²,

$$\underbrace{A(\sigma')}_{\text{proposed state}} \mid \underbrace{\sigma^{(i)}}_{\text{current state}} = A(\sigma'|\sigma) = \min (1, e^{-\beta\Delta E})$$

²Metropolis-Hastings is a generalised version of this acceptance criterion allowing for nonsymmetric proposal distributions

2.2.2 Metropolis algorithm

Substituting this acceptance rate into the general MCMC algorithm makes step 2 and 3 exact. We propose a new state picking a spin i uniformly at random and flipping it $\sigma_i \rightarrow -\sigma_i$. We then compute the resulting energy change $\Delta E = (E(\sigma') - E(\sigma^{(t)}))$ and accept the proposal with probability $A(\sigma'|\sigma^{(t)}) = \min(1, e^{-\beta\Delta E})$.

- if $\Delta E \leq 0$ then $e^{-\beta\Delta E} \geq 1$ and $A = 1$ and we always accept
- if $\Delta E > 0$ then $0 < e^{-\beta\Delta E} < 1$ and $A = e^{-\beta\Delta E}$, now we generate a random number $u \in [0, 1]$ and accept if $u > A$

All other steps stay the same.

An acceptance criterion that only ever accepted moves with $\Delta E \leq 0$ would be a greedy algorithm, *one which only considers the locally optimal solution*, and would get trapped in local minima without exploring the energy landscape. Allowing occasional uphill moves, according to temperature based probabilities, is exactly what allows the chain to escape such minima. This idea of temperature dependant acceptance is exactly what we will apply later in simulated annealing.

2.3 The Ising Model

Having derived the Metropolis algorithm we'll now apply it to the Ising model. The Ising model is defined by the Hamiltonian [1],

$$E = - \sum_{\langle ij \rangle}^N J_{ij} \sigma_i \sigma_j - \sum_i^N h_i \sigma_i$$

where $\langle ij \rangle$ sums over neighbouring spin pairs, J_{ij} is the coupling strength between spin $i \rightarrow j$ and h_i is the external field acting on the spin i .

The Metropolis algorithm requires us to calculate the energy difference ΔE for the proposed flip σ'

$$\begin{aligned} \sigma' : \sigma_k &\rightarrow -\sigma_k \\ \sigma'_k &= -\sigma_k \quad \sigma'_i = \sigma_i \text{ for } i \neq k \end{aligned}$$

our change in energy from the propose state σ' is

$$\begin{aligned} \Delta E &= (E(\sigma') - E(\sigma^{(t)})) \\ &= \left(-(-\sigma_k) \sum_{j \in \langle k \rangle}^N J_{kj} \sigma_j - h_k(-\sigma_k) \right) - \left(-\sigma_k \sum_{j \in \langle k \rangle}^N J_{kj} \sigma_j - h_k \sigma_k \right) \\ &= 2\sigma_k \left(\sum_{j \in \langle k \rangle}^N J_{kj} \sigma_j + h_k \right) \end{aligned}$$

Flipping single spins only affects its neighbours allowing us to have to not have to recompute the full energy each time, only the local sum ΔE of differences.

For a 2d square lattice we get four neighbours u, d, l, r ,

$$\Delta E = 2\sigma_k (J_{ku}\sigma_u + J_{kd}\sigma_d + J_{kl}\sigma_l + J_{kr}\sigma_r + h_k) \quad (2.1)$$

2.3.1 Implementing metropolis for a 2d lattice

With ΔE (2.1) we can implement the Metropolis algorithm for a 2D Ising lattice directly from the steps above: pick a random state, repeatedly propose single spin flips, accepting or rejecting according to $A(\sigma'|\sigma^{(t)}) = \min(1, e^{-\beta\Delta E})$.

The only other thing we need before we can build our functions is the concept of a *sweep*. If we run our metropolis algorithm only updating one single random flip per iteration we will end up with samples that leave most of the lattice the same and hence samples become highly correlated. To avoid this per iteration, *sweep*, we update $N \times N$ spin flips one per lattice site allowing on average each spin one chance to flip.

Every MCMC method that we build will follow this structure: first defining a sweep, then applying these sweeps some amount of times. The starting metropolis approach is build as such.

We need a sweep once function that will run our updates for one sweep and output the energy.

```
def sweep_once(. .)
    for _ in range(N * N):
        . .
    return E
```

Here we we need to calculate some random point (x, y) and calculate ΔE , this is what will change with different models. Where the modulo indexing $((x + 1) \% N)$ applies periodic boundary conditions, removing edge boundary conditions.

```
x = np.random.randint(N)
y = np.random.randint(N)
nn = (
    s[(x + 1) \% N, y] +
    s[(x - 1) \% N, y] +
    s[x, (y + 1) \% N] +
    s[x, (y - 1) \% N]
)
dE = 2 * s[x, y] * (J * nn + h)
```

Then we need to apply our acceptance rule, this step will stay the same throughout this project.

```
if dE <= 0 or np.random.rand() < math.exp(-dE / T):
    s[x, y] *= -1
    E += dE
```

This will give us a new energy sample for each sweep, with this we can build a 2d lattice metropolis sampling as such,

```
...
s = np.random.choice(np.array([-1, 1]), size=(N, N))
E = energy(s, J, h)
...
for _ in range(sweeps):
    E = sweep_once(s, N, J, h, T, E)
    energies.append(E)
...
```

2.3.2 Observables

Once the Markov chain has equilibrated and we've discarded the burn in rate(the section of the chain dependant on initial conditions), the remaining states are like samples approximately drawn from the Boltzmann target distribution,

$$p(\sigma) = \frac{e^{-\beta H(\sigma)}}{Z}.$$

Now that we have samples we can use them to estimate physical observables of our Ising model as sample averages using Monte Carlo sampling.

Average Energy. The factor $\frac{1}{N^2}$ accounts for the number of spins and the $\frac{1}{2}$ accounts for double counting each pair (ij) and (ji) .

$$\langle E \rangle = \frac{1}{N^2} \left(-\frac{1}{2} \sum_{ij}^N J_{ij} \sigma_i \sigma_j - \sum_i^N h_i \sigma_i \right) = \text{average } E$$

Average magnetisation, the mean spin value across the system.

$$\langle m \rangle = \frac{1}{N^2} \sum_i \sigma_i = \text{average } \sigma$$

Specific heat capacity. from thermodynamics we know

$$c_v = \left. \frac{\partial E}{\partial T} \right|_v$$

Statistical mechanics also gives us a definition of average energy,

$$\langle E \rangle = -\frac{\partial \ln Z}{\partial \beta}$$

so we can relate the two.

$$\begin{aligned}
c_v &= - \left. \frac{\partial}{\partial T} \frac{\partial \ln Z}{\partial \beta} \right|_v \\
&= - \left. \frac{\partial \beta}{\partial T} \frac{\partial}{\partial \beta} \frac{\partial \ln Z}{\partial \beta} \right|_v \\
&= - \left. \frac{\partial}{\partial T} \left(\frac{1}{k_B T} \right) \frac{\partial^2 \ln Z}{\partial \beta^2} \right|_v \\
&= \frac{1}{k_B T^2} (\langle E^2 \rangle - \langle E \rangle^2) = \frac{\text{Variance } E}{k_B T^2}
\end{aligned}$$

The specific heat capacity is therefore directly proportional to the variance of the energy samples, allowing us to calculate c_v without taking any derivatives.

Magnetic susceptibility. By an analogous argument,

$$\chi = \lim_{B \rightarrow 0} \frac{\partial \langle m \rangle}{\partial B} = \dots = \frac{N^2}{k_B T} (\langle m^2 \rangle - \langle m \rangle^2) = \frac{N^2 \times \text{Variance } m}{k_B T}$$

Similarly we have magnetic susceptibility proportional to variance and no derivatives are required.

With these four equations [2] we can now calculate our observables from averages and variances from a sample of energies outputted from the metropolis Hastings.

```

Nspins = N * N
E = np.mean(energies) / Nspins
cv = np.var(energies) / (T**2 * Nspins)
m = np.mean(np.abs(mags)) / Nspins
chi = np.var(mags) / (T * Nspins)

```

Where we've added an absolute value in the magnetisation since below T_c the system spontaneously magnetises in either positive or negative direction with equal probability adding to zero even if each m is strongly magnetised, $|m|$ is therefore the physically relevant quantity.

We can now run these algorithms on a square lattice such as a 20×20 , which can be seen in Figure 2.1.

After running metropolis method with parameters: $sweeps = 10000$ and $warmup = 8000$. We reproduced the literature results in Figure 2.2. We found that early on with $warmup \approx 2000$ we were getting $\langle E \rangle$ and $\langle m \rangle$ being their correct shapes [2], however c_v and χ had drastic error bars. We discovered that we needed to disregard many more early samples or else the samples were too highly correlated and making our variance observables blow up.

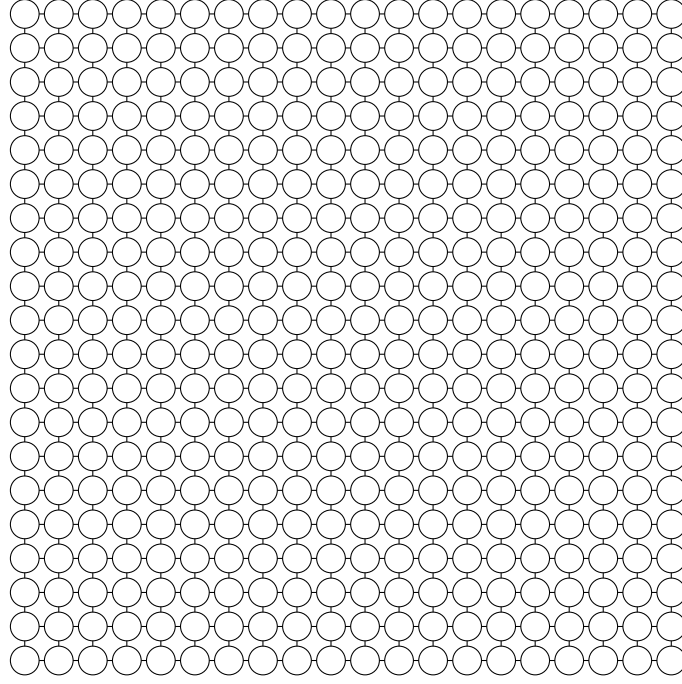


Figure 2.1: Square Lattice (20×20)

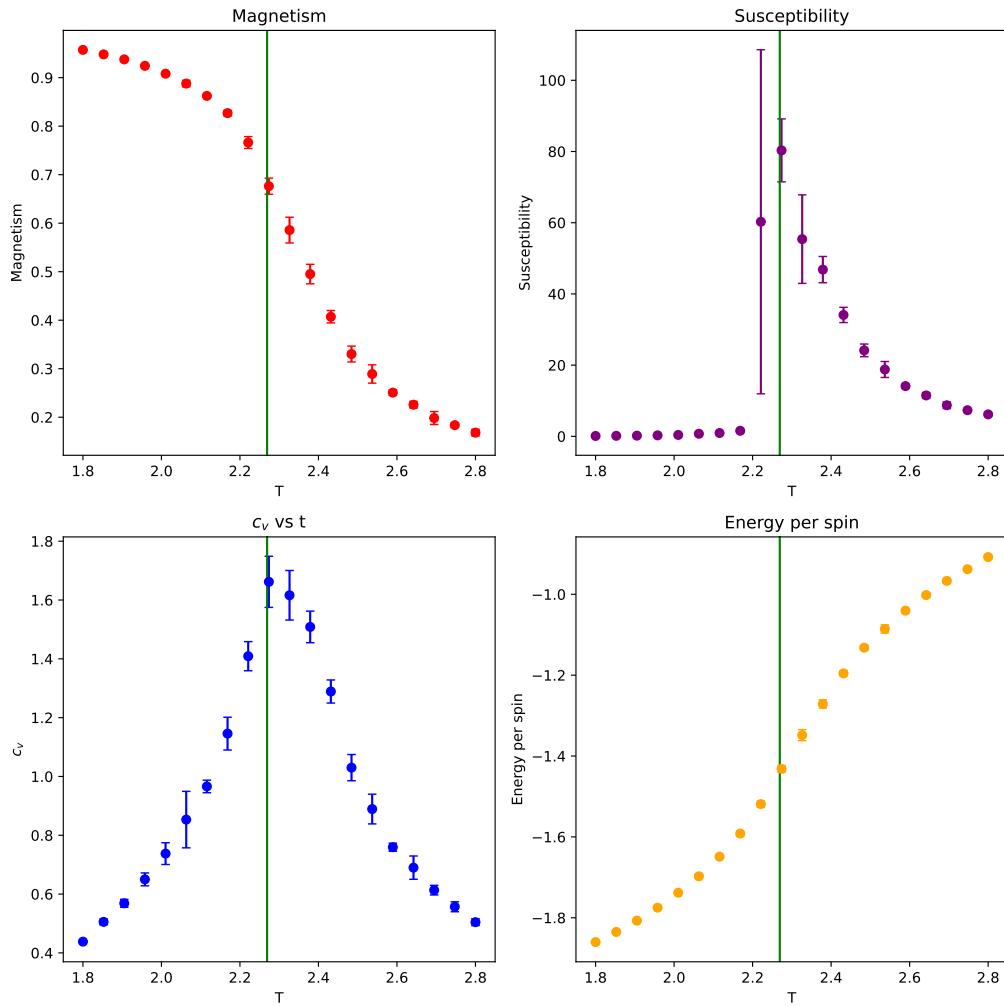


Figure 2.2: Metropolis sampling of observables ($\langle m \rangle, c_v$ & $\langle E \rangle$) across many temperatures ($sweeps = 10000$, $warmup = 8000$, for a $N \times N$ 2D lattice $N = 20$)

2.4 Simulated Annealing

The Metropolis algorithm samples the equilibrium distribution at a fixed temperature T , but with many of our problem all we need is the ground state energy of the system rather than a full sample of the equilibrium distribution. We can adjust the temperature T over a run to optimise the run and minimise the system with less sweeps.

The name coming from analogy to metallurgy, at hot temperatures atoms move around freely and as the material cools they settle into low energy configurations. Cooling slowly gives the system time to explore the full energy landscape finding the true global minima, the optimal crystal structure. However cooling too quickly risks the system getting stuck in a local minima of the energy landscape, a metastable defect state. This is the idea we apply to our computational methods.

Returning to our metropolis acceptance rate $A(\sigma'|\sigma^{(t)}) = \min(1, e^{-\beta\Delta E})$, where $\beta = 1/T$. As T increases β falls and the system accepts more *uphill* spin flips letting it escape minima, whereas when T decreases β rises and the system rarely accepts *uphill* spin flips, near $T = 0$ essentially none are accepted. This mechanism of starting the chain very hot, at T_{Start} , lets the system explore broadly without getting trapped in local minima clustering near the global minima as we slowly lower the temperature, to T_{End} , refining the chain to the global minima rather than the first one it finds.

We use *geometric annealing* as our cooling schedule, this is where the temperature is reduced by a constant factor $\alpha \in (0, 1)$ after some number of sweeps at each temperature. Adjusting α will be how we can play around with annealing speeds in problems.

$$T \rightarrow \alpha T$$

2.4.1 Implementing annealing

The structure of this will look very similar to the metropolis methods build and discussed above, such as initialising the same was and calling the same sweep function.

For brevity, we will only discuss the difference, the annealing schedule. At each temperature we perform a number of sweeps similar to last time, but now we decrease our temperature $T \rightarrow \alpha T$ each time.

```
...
T=Tstart
while T > Tend:
    for _ in range(sweeps_per_T):
        E = sweep_once(s, N, J, h, T, E)
        energies.append(E)
    T *= alpha
...
```

This will then output an array of the energy at each temperature with

```
energies[-1]
```

giving us the final and minimum energy of the system.

2.4.2 Generalising: Spin Glass Theory

So far we've been dealing with Ising models where J_{ij} is a single constant across all neighbours $\langle ij \rangle$ and h_i is a single constant for the external field. Therefore, our energy of an Ising model can be wrote in two forms

$$H = -J \sum_{\langle ij \rangle} \sigma_i \sigma_j - h \sum_j \sigma_j$$
$$H = - \sum_{\langle ij \rangle} J_{ij} \sigma_i \sigma_j - \sum_j h_j \sigma_j$$

The natural generalisation is allowing J_{ij} to vary between pairs and similarly h_i . This is the *spin glass model* where now J is an $N \times N$ matrix and h is a n vector. This is a small change mathematically with the Hamiltonian being unchanged, however this becomes computationally more difficult with each spins local field being computed as a dot product.

We need to adjust our sweep function accordingly,

```
def sweep_once(s, J, h, T, E):
    n=len(s)
    for _ in range(n):
        i = np.random.randint(n)
        dE = 2 * s[i] * (J[i] @s + h[i])

        if dE <= 0 or np.random.rand() < math.exp(-dE / T):
            s[i] *= -1
            E += dE
    return E
```

This spin glass form is what makes this Ising formulation applicable for combinatorial problems, any energy function we can express in a quadratic form over spin (or as we'll see next binary) variables can be optimised using the same annealing framework we've already built.

2.4.3 Optimisation: JIT Compilation with Numba

This generality that allows us to express more problems, allowing spins to interact with every other spin with varying weights comes at a computational cost. We're moving away from a 2d lattice where interactions are simplified to 4 neighbours, to a fully general calculate of the local field contribution a sum over all N variables. This also increases a sweep from $O(N)$ to $O(N^2)$ which quickly becomes a problem for large lattice sizes in python.

To tackle this we employ Numbas **just in time** compiler, this decorator, `@njit`, wraps around python functions and compiles them into machine code the first time its called. This can break more complicated functions but since our sweep and energy difference functions are just simple numerical loops it works fine. Without changing anything in the algorithm just wrapping the sweep and ΔE function we see substantial speed ups, before implementing this the code was borderline unusable.

2.4.4 Spin to binary: QUBO

Many optimisation problems that we will deal with later will be naturally stated in binary $x_i \in \{0, 1\}$ rather than $s_i \in \{-1, 1\}$. This can be done with the transformation,

$$\begin{aligned}x_i &\rightarrow \frac{1 + s_i}{2} \\s_i &= 2x_i - 1\end{aligned}$$

Imputing this into the spin glass Hamiltonian and rearranging gives us

$$E = \mathbf{x}^T Q \mathbf{x}$$

where $\mathbf{x}$ is our vector of binary variables and Q absorbs J_{ij} and h_i into a weighting matrix. This form is called Quadratic Unconstrained Binary Optimisation (QUBO), this is the standard form for our combinatorial optimisation problems when dealing with these *Ising solvers*. Unconstrained refers to the fact there is no constraint on $\mathbf{x}$ other than it being binary. Much of our work later will be in constructing our problems in terms of constraints in Q for QUBO energy functions but for now we will update our annealing methods to accept binary variables.

We need to adapt our Metropolis sweep to

```
...
for _ in range(N):
    i = np.random.randint(N)
    dE = delta_energy(x, Q, i)
    if dE < 0 or np.random.rand() < np.exp(-dE / T):
        x[i] = 1 - x[i]
        E += dE
...
```

and adjust our energy difference for flipping a bit.

With our binary flips x_i and $1 - x_i$ they aren't negatives of each other like flips before with σ and $-\sigma$, hence we need to split up the two cases. Then we expand out our energy function,

$$\begin{aligned}E &= \mathbf{x}^T Q \mathbf{x} \\&= Q_{ii}x_i + 2x_i \sum_{j \neq i} Q_{ij}x_j \\&= Q_{ii}x_i + 2s_i x_i\end{aligned}$$

Where we define s_i the interactions with every other bit to be

$$s_i = \sum_{j \neq i} Q_{ij}x_j$$

```

@njit
def delta_energy(x, Q, i):
    # compute interactions with every other bit
    s = 0.0
    N = len(x)
    for j in range(N):
        if j != i:
            s += Q[i, j] * x[j]

    if x[i] == 0:
        # 0 -> 1
        return Q[i, i] + 2.0 * s
    else:
        # 1 -> 0
        return -(Q[i, i] + 2.0 * s)

```

2.5 Improving the annealer

2.5.1 sweeping until converged

The final improvements we wrote for our annealer were useful later on when we applied QUBO to rocket optimal control where the scale of binary variables blows up quickly with encoded continuous variables.

The main idea is to have variable sweeps per temperature rather than fixing to some constant like before. We will now have a rolling array of *recent energies* that holds the last *window* of energy values at the given temperature. The loop will do a sweep each time and record the energy, cycling through the array with $s \% \text{window}$. This leaves *recent_energies* holding the most recent *window* of energies regardless of sweeps ran.

```

def sweep_until_converged(x, Q, T, E, window=50, tol=1e-3, max_sweeps=5000):
    ...
    recent_energies = np.zeros(window)
    for s in range(max_sweeps):
        E = sweep_once_QUBO(x, Q, T, E)
        recent_energies[s % window] = E

```

The next loop is the convergence check, to start $s \geq \text{window}$ ensures we aren't comparing with left over zeros. Next is a relative tolerance check, since energy magnitudes vary greatly between problems, breaking when the sweeps are no longer finding lower energy states and our spread shrinks.

```

if s >= window:
    spread = recent_energies.max() - recent_energies.min()
    if spread < tol * abs(E):
        break

```

This method removes wasted sweeps per temperature where the system has already found the lowest energy at that temperature but it still must sweep x times more. Not only does this remove the question *how many sweeps per temperature are needed for it to equilibrate?* but also speeds the system up

drastically.

The annealing algorithm itself doesn't require many changes from before.

```
...
T = T_start
while T > T_end:
    E, s_done = sweep_until_converged(x, Q, T, E, window, tol, max_sweeps)

    energies.append(E)

    T *= alpha
...
```

2.5.2 More effective starting and ending temperatures

Originally, when we implemented this we treated T_{Start} and T_{End} as parameters however we found this was both ineffective and broke down when the scale of one flips dE grew. This makes sense with definition of probability,

$$p \propto e^{-dE/T}$$

inverting this relation gives

$$T \propto \frac{-dE}{\ln p}$$

hence the ratio $\frac{dE}{T}$ matters. Now instead we insert cold and hot probability rates that we require rather than temperatures we require. This is both much faster as it doesn't waste time sweeping too hot or too cold.

The idea around the *estimator* is to sample many random configurations of σ computing the energy change dE for the proposed flip, and use the resulting distribution to set T_{Start} and T_{End} .

```
...
dE_typical = np.percentile(all_dEs, 50)    # median dE, for T_start
dE_small   = np.percentile(all_dEs, 5)     # a small dE, for T_end

T_start = -dE_typical / np.log(hot_accept_prob)
T_end   = -dE_small  / np.log(cold_accept_prob)
...
```

2.5.3 Comparison: dwave

Finally we compared our convergence based annealer with D-Wave's *neal* simulated annealer package and D-Wave's *tabu* search. Using our 1-D rocket trajectory problem, that we'll derive later, as the QUBO energy function we ran 1500 independent runs for each solver. We see in Figure 2.3 that both our custom annealer and *neal* perform similarly converging to $\approx -790,000$ to $-785,000$. The main difference between the two annealers was the run time with our custom annealer performing this in

≈ 10 mins and D-wave's package performing this in 30s, which is quite a drastic improvement on our annealing but the accuracy of our annealer shows it's a correct methodology. The tabu sampler was also compared to show another approach, however it converged to a higher and worse energy on average with a broader spread, indicating its a worse too for this task.

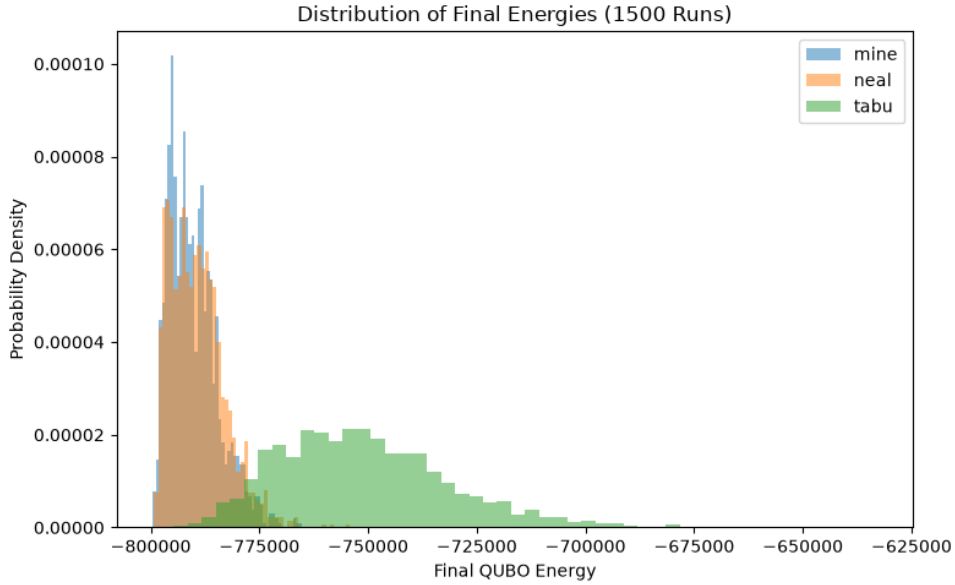


Figure 2.3: Comparison of D-wave's neal & tabu vs our custom annealer annealer

3 Constructing Energy functions

3.1 The General Structure

In order to successfully convert a constrained optimisation problem into a form that we can use our computational annealing framework with we need to build a Spin or QUBO Hamiltonian that encodes the problem. These Hamiltonians or as we'll call them interchangeably *energy functions* are built such that the minima of the function's energy landscape is the optimal solution of the problem.

This mirrors a structure you might already recognise that of enforcing constraints via Lagrangian multipliers. The language of Lagrangian multipliers takes a constrained problem such as,

$$\min f(x) \quad \text{s.t} \quad g(x) = a$$

Then expresses the problem in an unconstrained Lagrangian form of

$$\min f(x) + \lambda(g(x) - a)^2$$

where we've penalised any deviations from our constraint of $g(x) = a$.

We'll use this idea when building our energy functions. Now typically we will have some energy function consisting of two components,

$$H = H_A + H_B$$

Where H_A will be used to enforce the problems constraints whilst H_B will encode the objective we want to maximise or minimise.

Constraints are written as a sum of squared terms, each constraint will be in the form

$$H_A = A \left(\sum_i g_i(\vec{x}) - a_i \right)^2$$

where we have $g_i(\vec{x}) = a_i$ as our constraints. This will similarly punish any deviation from the true valid solution following the constraints are the ground state that of $H_A = 0$.

The other important thing we will see throughout is that the ratio between our hamiltonians coefficients A and B matters. If A is chosen to be too small relative to B the *minimum energy* might violate the problems constraints to prefer minimising the objective, hence producing a global minima that doesn't satisfy the problem. Therefore we need a condition comparing the ratios of A and B that guarantees we always ensure the constraints are satisfied before we deal with the objective.

To build up the logic of building these energy functions we will work through a sequence of combinatorial problems from *Ising formulations of many NP problems* by A.Lucas [3]. We will look at encoding techniques that progressively become more complex, from a singular penalty term to multi index variables and ordering constraints, which we will need later for our rocket trajectory formulation in the next section.

We'll start with partitioning problems, where we given some set of terms that we must divide into two subsets given constraints. These are a natural class of problem to start with as we can apply existing spin glass theory once we classify the two partitioned groups with $s_i = \pm 1$.

3.2 Number Partitioning

We'll start with *number partitioning* as unlike later problems, we can express this problem as a single penalty term H_A . We'll go slowly though this first exercise problem and speed through later ones. Our problem asks,

Given a set of N positive numbers $S = \{n_1, \dots, n_N\}$, is there a partition of numbers into two disjoint subsets A and B s.t. the sum of their elements equal?

This gives us a nice start a balancing condition, to set up our problem we assign each element $n_i \in S$ a spin $s_i \in \{-1, 1\}$ denoting which partition it belongs to.

$$\begin{cases} s_i = +1 \rightarrow \text{group A} \\ s_i = -1 \rightarrow \text{group B} \end{cases}$$

The construction will then sum each element with its corresponding spin which will spilt up accordingly

$$\sum_i^N n_i s_i = \sum_{\text{group A}} n_i - \sum_{\text{group B}} n_i$$

If our set S can be perfectly balanced between the sums of set A and set B then their sum will balance to zero, $\sum_i^N n_i s_i = 0$. This is the construction of our constraint in terms of spin variables. The last step of writing our energy function is squaring it punishing deviations from the optimal minimum solution.

$$\begin{aligned} H &= -A \left(\sum_i^N n_i s_i \right)^2 \\ &= -A \underbrace{\sum_{i=j}^N n_i^2}_{\text{constant}} - 2A \sum_{i<j}^N n_i n_j s_i s_j \end{aligned}$$

Since our aim throughout the project is minimising energy functions we are able to drop any constant terms in the construction as they won't affect which state will minimise $E(x)$, only adding a constant shift.

This gives us our energy cost function encoding the number partitioning problem,

$$E(x) = -2A \sum_{i<j}^N n_i n_j s_i s_j$$

Now to implement this in our spin glass theory computational framework we want to express this in terms of the coupling matrix elements J_{ij} and external field elements h_i , matching the form $H = -\sum_{\langle ij \rangle} J_{ij} s_i s_j - \sum_j h_j s_j$.

One thing to note is the two conventions when building our coupling matrix J . The symmetric matrix construction sets $J_{ij} = J_{ji} = \frac{1}{2}$ coefficients of $s_i s_j$ which requires us to introduce a $\frac{1}{2}$ to our off diagonals or the upper triangle construction where $J_{ij} =$ coefficients of $s_i s_j$ and $J_{ji} = 0$. Both are valid constructions of the problem they just require different implementations, we found it was easier to work with the symmetric construction in our code however we know a lot of the literature like the upper triangle construction since its slightly cleaner. We will continue with a symmetric construct throughout this project where not explicitly specified.

This leaves us with our first Ising formulation,

$$\begin{aligned} J_{ij} &= -A n_i n_j \\ h_i &= 0 \end{aligned}$$

with this construction of $E(x)$ in terms of J_{ij} and h_i we can now verify this with our existing computational framework we've built.

3.2.1 Implementing Number Partitioning

The structure of implementing these problems into python will be,

1. Build the problems energy function as an array
2. Anneal
3. Read the results and check validity of our annealed solution
4. Plot where applicable

Our number partitioning problem has a fairly straight forward implementation of building J as it can be neatly expressed as an outer product of two arrays, $(\vec{a} \otimes \vec{b})_{ij} = a_i b_j$.

```
def make_partition_ising(n, A=1.0):  
    n = np.asarray(n)  
    J = -2 * A * np.outer(n, n)  
    np.fill_diagonal(J, 0.0)  
    h = np.zeros(len(n))  
    return J, h
```

With these arrays we can apply our spin glass annealing and get the the minimal energy spin configuration of the problem.

```
_, _, spins = sim_annealing(J, h, alpha=0.99)
```

Lastly, we check whether our solution satisfies the problem.

```
setA = numbers[spins == 1]  
setB = numbers[spins == -1]
```

One thing to note is that not many sets will have valid solutions ($\sum_{i \in A} n_i - \sum_{i \in B} n_i = \Delta n = 0$), in these cases the set up will find a solution that minimises the remainder ($\min \Delta n$).

Now giving our construction a set S that we know has valid solutions,

$$S : [1, 3, 1, 2, 2, 1]$$

Annealing will give us two sets that satisfy our constraint,

$$\begin{array}{ll} A : [3, 2] & \sum a_i = 5 \\ B : [1, 1, 2, 1] & \sum b_i = 5 \end{array}$$

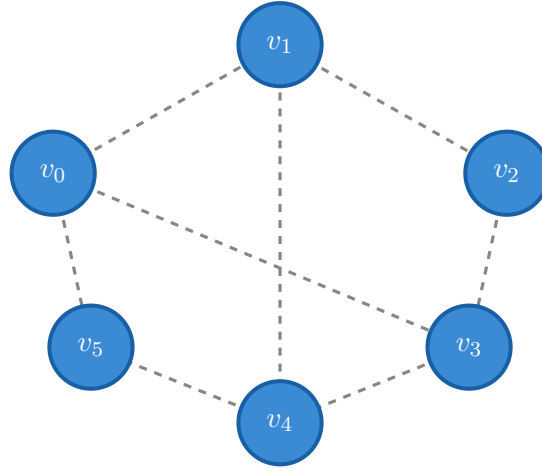
Now that we have a concrete example of the process we can move on to more interesting examples with visualisations and objective terms.

3.3 Graph Partitioning

Moving on to a problem where we introduce an objective term H_B that we are trying to minimise alongside the constraints. This problem asks,

Given some undirected graph $G(V, E)$, where $v \in V$ are vertices and $e \in E$ are edges, with an even number of edges $N = |V|$. We ask, what partition of V into two equally sized subsets will minimise the number of edges connecting the two subsets?

This is a nice problem as we visualise the problem with graphs, $G(V, E)$, such as below where we have a 6 node cycle graph with two extra edges $(0, 3)$ and $(1, 4)$.



Similarly to the last problem, to balance the two subsets into two subsets with an even amount of vertices we assign each partition a spin $s_i = \pm 1$, summing over these signed nodes now produces a minima when each subset has $N/2$ vertices. This deals with the constraints of the problem,

$$H_A = A \left(\sum_i^N s_i \right)^2$$

Now we introduce the new idea for graph partitioning, an objective component H_B . Here we will minimise the number of edges connecting the two subsets. We want to penalise our energy function for each edge it has connecting the two subsets, this will be labelled with $(ij) \in E$. Now consider the different cases of edges as we have a combination of two spins $v \in V$, they agree $\uparrow\uparrow$ & $\downarrow\downarrow$ or they both disagree $\downarrow\uparrow$ or $\uparrow\downarrow$.

Hence we construct a fraction penalising our energy function if the two vertices i and j have different spins,

$$\frac{1 - s_i s_j}{2} = \begin{cases} \text{when } s_i = s_j = 1 & \implies \frac{1 - s_i s_j}{2} = 0 \\ \text{when } s_i = s_j = -1 & \implies \frac{1 - s_i s_j}{2} = 0 \\ \text{when } s_i = \pm 1, s_j = \mp 1 & \implies \frac{1 - s_i s_j}{2} = 1 \end{cases}$$

This gives us our objective function as we want the configuration that will minimise this quantity, now we can express our objective as H_B

$$H_B = B \sum_{(i,j) \in E} \frac{1 - s_i s_j}{2}$$

Combining these gives us our energy function for graph partitioning,

$$\begin{aligned} H &= H_A + H_B \\ &= A \left(\sum_i^N s_i \right)^2 + B \sum_{(i,j) \in E} \frac{1 - s_i s_j}{2} \end{aligned}$$

Once again we need to expand out our energy function to express it in a Ising form.

$$\begin{aligned} H &= A \left(\sum_i^N s_i \right)^2 + B \sum_{(i,j) \in E} \frac{1 - s_i s_j}{2} \\ &= A \sum_i^N s_i^2 + 2A \sum_{i>j}^N s_i s_j + \frac{B}{2} \sum_{(i,j) \in E} -\frac{B}{2} \sum_{(i,j) \in E} s_i s_j \\ &= \underbrace{AN}_{\text{constant}} + \underbrace{\frac{B|E|}{2}}_{\text{constant}} + 2A \sum_{i>j}^N s_i s_j - \frac{B}{2} \sum_{(i,j) \in E} s_i s_j \\ H &= 2A \sum_{i>j}^N s_i s_j - \frac{B}{2} \sum_{(i,j) \in E} s_i s_j \end{aligned}$$

For these last two cases it has been clean enough to expand our our energy function into linear and quadratic terms however we'll see that later this becomes a substantial part of the difficulty in formulating our problem. Similarly to last time we are left with h_i , however now we have two terms for our coupling matrix J now depending on if the edge is part of the subset of edges E which connect the two subsets of vertices.

$$\begin{aligned} J_{ij} &= \begin{cases} -A & \text{if } (i,j) \notin E \\ -A - B/4 & \text{if } (i,j) \in E \end{cases} \\ h_i &= 0 \end{aligned}$$

Our final difference with this construction is the importance of the constraint on A and B . Lucas derives a constraint on our coefficients based on ensuring the minimum energy penalty gained for violating the partition constraint always exceeds the maximum energy gain for flipping one spin in the objective term. This idea will continue to appear in these constructions: a constraint on the coefficients will ensure the ground state found corresponds to a valid, constraint satisfying state.

$$\frac{A}{B} \geq \frac{\min(2\Delta, N)}{8} \quad \text{where } \Delta = \text{degree of } G$$

3.3.1 Implementation Graph Partitioning

This implementation follows the same building structure as we saw with number partitioning, building Q annealing check. With our main difference being our use of *networkx* allowing us to work with graphs $G(V, E)$ as an array of edges and nodes. To build a graph such as the *six node cycle graph with two extra edges* (0, 3) and (1, 4), the graph above (3.3), we have

```
G = nx.cycle_graph(6)
G.add_edge(0, 3)
G.add_edge(1, 4)
```

With our graphs encoded as arrays we can implement the two cases, whether an edge connects two vertices or not, into our Q construction.

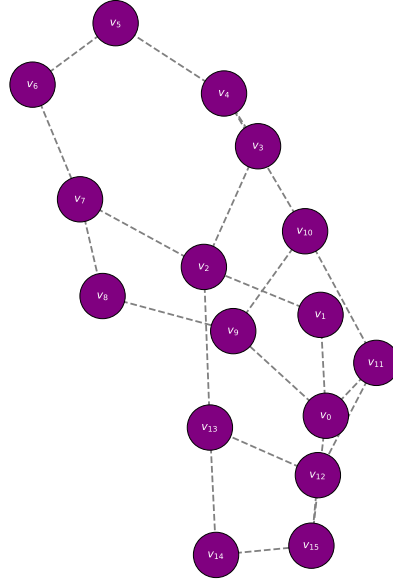
```
...
N = G.number_of_nodes()
nodes = list(G.nodes())
...
idx = {v:i for i,v in enumerate(nodes)}
for u,v in G.edges():
    i, j = idx[u], idx[v]
    J[i,j] += B/4.0
    J[j,i] += B/4.0
...
```

After annealing, we recover the two subsets from the sign of each spin and identify which edges connect the two sets with *connecting_edges*.

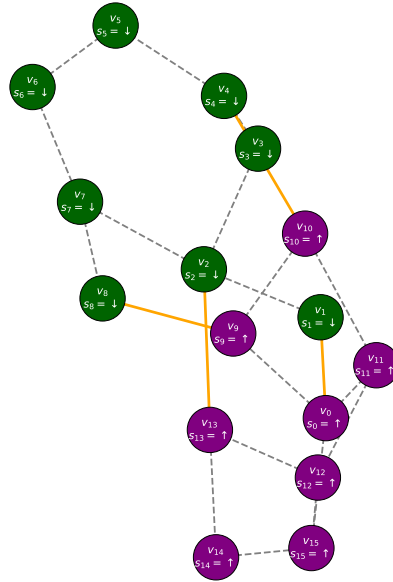
```
setA = [nodes[i] for i in range(len(s)) if s[i] == 1]
setB = [nodes[i] for i in range(len(s)) if s[i] == -1]
connecting_edges = [(u,v) for u,v in G.edges() if (u in setA) != (v in setA)]
```

Validity here requires checking that both subsets have equal vertices $|A| = |B| = N/2$ and then calculating the amount of edges connecting the two subsets the objective value we were minimising.

We wrote a function to randomise a cycle graph of a random even number of vertices, with a handful of extra random edges added. This is then applied to graph partitioning method above Figure 3.1 for $N = 16$ with added 6 extra edges where we found a valid solution to the problem with annealing.



(a) Original randomised cycle graph $N = 16$ with added 6 extra edges



(b) Annealed randomised cycle graph $N = 16$ with added 6 extra edges

Figure 3.1: Graph partitioning for a cycle graph $N = 16$ with added 6 extra edges

3.4 Graph Colouring

The problems so far have used spin variables $s_i \in \{-1, +1\}$, natural for partition problems. The rest of the problems will instead be constructed with binary variables $x_i \in \{0, 1\}$. This swap is a more natural way of expressing the problem as we'll see a vertex is either assigned a colour or not, rather than splitting it into two groups. Our energy functions will now move to QUBO formulation with a Q coupling/weighting matrix rather than an Ising formulation of J_{ij} and h_i .

Graph colouring's other main addition is two index variables $x_{v,i}$, which will be used throughout this project. The problem asks

Take some undirected graph $G(V, E)$ and n colours. Can we colour each vertex such that no edge connects vertices of the same colour?

This is a generalisation of the famous how many colours are needed to colour a map such that no two countries of the same colour share a border, which is rigorously shown to be $n \geq 4$. [4].

Here we've introduced two index binary variables $x_{v,i}$, $v \in V$ is a vertex and $i \in \{1, \dots, n\}$ indexes the available colours.

$$x_{v,i} = \begin{cases} 1 & \text{if vertex } v \text{ has colour } i \\ 0 & \text{otherwise} \end{cases}$$

Now we'll construct our constraints, firstly we need a term that enforces that each vertex has only one colour

$$H_{A,1} = A \sum_v \left(1 - \sum_{i=1}^n x_{v,i} \right)^2$$

Now the second constraint needs to penalise one colour i connecting two vertices u and v .

$$H_{A,2} = A \sum_{(u,v) \in E} \sum_{i=1}^n x_{u,i} x_{v,i}$$

This gives us our multi indexed variable energy function,

$$H = A \sum_v \left(1 - \sum_{i=1}^n x_{v,i} \right)^2 + A \sum_{(u,v) \in E} \sum_{i=1}^n x_{u,i} x_{v,i}$$

Now we need to expand out our components into a QUBO form we can read off for the Q matrix. Our first constraint $H_{A,1}$,

$$\begin{aligned} H_{A,1}A &= \sum_v \left(1 - \sum_i x_{v,i} \right)^2 \\ &= A \sum_v \left(\underbrace{1}_{\text{drop}} - \sum_i x_{v,i} + \left(\sum_i x_{v,i} \right)^2 \right) \\ &= A \sum_v \left(- \sum_i x_{v,i} + 2 \sum_{i < j} x_{v,i} x_{v,j} \right) \end{aligned}$$

and then $H_{A,2}$ is already quadratic, $A \sum_{(u,v) \in E} \sum_{i=1}^n x_{u,i} x_{v,i}$

$$H = A \sum_v \left(- \sum_i x_{v,i} + 2 \sum_{i < j} x_{v,i} x_{v,j} \right) + A \sum_{(u,v) \in E} \sum_{i=1}^n x_{u,i} x_{v,i}$$

$$Q_{(v,i),(v,i)} = -A$$

$$Q_{(v,i),(v,j)} = Q_{(v,j),(v,i)} = A \quad \text{for } i \neq j \text{ (same vertex, different colours)}$$

$$Q_{(u,i),(v,i)} = Q_{(v,i),(u,i)} = \frac{A}{2} \quad \text{for } (u,v) \in E \text{ (connected vertices, same colour)}$$

3.4.1 Implementing Graph Colouring

The implementation follows the same structure as before, the only real difference is the flattening out of our two indices,

```
def idx(v, i, n):
    return v * n + i
```

With this we can then build Q with a few for loops once we also index our nodes accordingly

```
...
nodes = list(G.nodes())
node_idx = {v: i for i, v in enumerate(nodes)}
...
```

Annealing Q and then we can check our validity with two constraints. First by making an array of colours assigned from our annealed $x_{v,i}$ and then checking this array's length is equal to one, then for the second constraint we need to check no two edges have vertices that are the same colour.

A nice version of the graph colouring problem we made: generates a random cycle graph, adds a few random connections, then starts a loop with the number of colours $n = 1$, it anneals the problem checks validity, if it fails the validity check increases colours by one and repeats until we get a valid solution plotting it. An example of one of these randomised graphs can be seen below in Figure 3.2

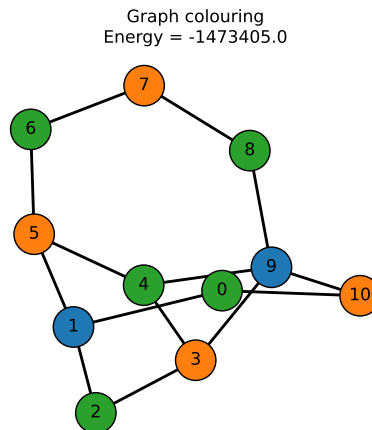


Figure 3.2: Graph Colour

3.5 Hamiltonian Cycle/Path

Building on the two index variables $x_{v,i}$ from graph colouring however now we will implement i as a *position in a sequence* rather than an on or off.

This is a generalisation of the famous *Seven Bridges of Königsberg* which asked could you make it around the 7 bridges of the city without crossing the same bridge twice. Euler famously proved no such walk was possible for the bridges of Königsberg, we'll instead look at not crossing vertices rather than edges.

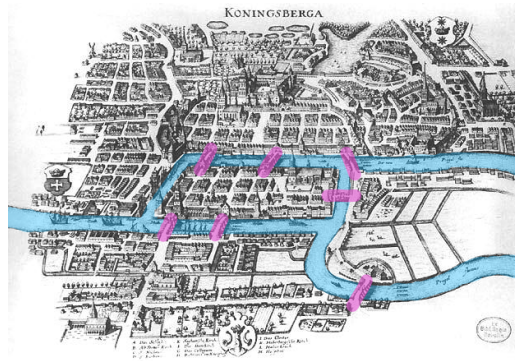


Figure 3.3: Bridges of Königsberg illustrating the Seven Bridges of Königsberg problem.
Image by Twotwos, licensed under CC BY-SA 4.0 via Wikimedia Commons [5].

The Hamiltonian Path problem generalises this as

Take some graph $G(E, V)$ (directed or undirected) with $N = |V|$. Can we start at one vertex and visit every other vertex in the graph without returning to any vertex twice? If we enforce the sequence returns to the start we get a *Hamiltonian cycle*.

We'll just look at Hamiltonian cycles as the only difference is summing from to N or $N - 1$ in our loops.

Now we label the vertices $v \in V$, $(1, 2, \dots, N)$, and the edges $(uv) \in E^3$. Our binary bits are $x_{v,i}$ where $v \in V$ is our vertex and i is position in the cycle order. We will only have H_A , however we will have three components, $H = H_1 + H_2 + H_3$, and will look similar to previous functions we've looked at.

H_1 and H_2 will have exactly the same structure as graph colouring's $H_{A,1}$, with each vertex having one colour swapped to enforce *each vertex having one position in the cycle* and *each position in the cycle having one vertex*. Finally, H_3 is our only new idea penalising consecutive positions that aren't connected in the cycle.

³the order of (uv) matters for directed edges, however we can extend the ideas to undirected graphs by adding (vu) whenever (uv) is added to E

$$\begin{aligned}
H_1 + H_2 &= A \sum_{v=1}^n \left(1 - \sum_{j=1}^N x_{v,j} \right)^2 + A \sum_{j=1}^N \left(1 - \sum_{v=1}^n x_{v,j} \right)^2 \\
&= A \sum_{v=1}^n \left(- \sum_i x_{v,i} + 2 \sum_{i < j} x_{v,i} x_{v,j} \right) + A \sum_{j=1}^N \left(- \sum_{v=1}^n x_{v,j} + 2 \sum_{u < v} x_{u,j} x_{v,j} \right)
\end{aligned}$$

The idea around H_3 is to penalise consecutive positions that aren't in the cycle, we take some $(u, v) \notin E$, then we want to penalise $x_{u,j}$ and $x_{v,j+1}$ as they aren't connected in our cycle.

$$H_3 = A \sum_{(uv) \notin E} \sum_{j=1}^N x_{u,j} x_{v,j+1}$$

Combing this we get our energy cost function for a Hamiltonian cycle,

$$\begin{aligned}
H_A &= H_1 + H_2 + H_3 \\
H_A &= A \sum_{v=1}^n \left(- \sum_i x_{v,i} + 2 \sum_{i < j} x_{v,i} x_{v,j} \right) + A \sum_{j=1}^N \left(- \sum_{v=1}^n x_{v,j} + 2 \sum_{u < v} x_{u,j} x_{v,j} \right) + A \sum_{(uv) \notin E} \sum_{j=1}^N x_{u,j} x_{v,j+1}
\end{aligned}$$

3.5.1 Implementing Hamiltonian Cycle

The implementation of the energy function will follow the same flattened two index approach as graph colouring, now with (vertex, position) rather than (vertex, colour). Building Q requires three separate loop sections, one for each H_1 , H_2 & H_3 as each penalise different pairs of indices.

Checking validity also mirrors graph colouring's approach for the first two constraints, as we were checking there was one colour for each vertex and now we're checking *each vertex appears once* and *each position is filled once*. We build an arrays for recoding, checking the length of eah array is one, our arrays are: for each vertex, which positions are defined and for each position, which vertices are defined. The third constraint, that consecutive positions in the cycle are actual edges in G , is checked by walking though the cycle order and ensuring it matches G .

```

if valid:
    for j in range(N):
        u = cycle[j]
        v = cycle[(j + 1) % N]
        if not G.has_edge(u, v):
            missing_edges.append((u, v))
    if missing_edges:
        valid = False

```

Here $(j + 1) \% N$ ensures that the cycle wraps around and N connects with the start.

We reuse the same random cycle plus a extra edges graph generator from for graph partitioning to make our first tests of this construction, shown below for $N = 9$ in Figure 3.4.

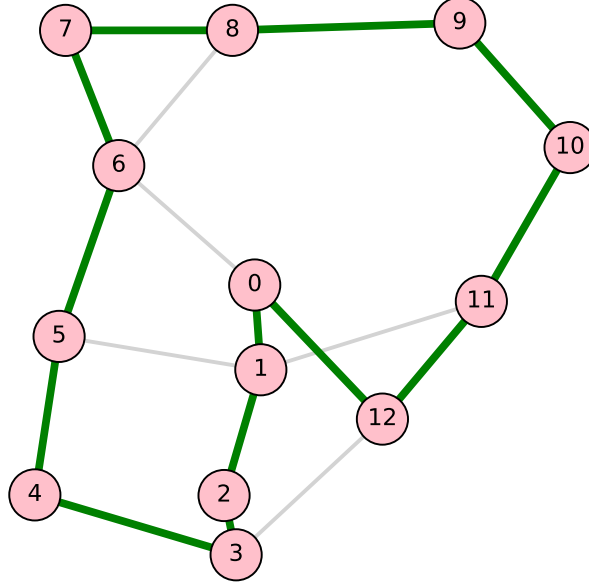


Figure 3.4: Hamiltonian Cycle of a complete graph $N = 9$ with added 4 extra edges.
Cycle: [8, 0, 1, 2, 3, 4, 5, 6, 7]

3.6 Travelling Salesman

Putting it all together, we get the traveling salesman problem (TSP), combining the flattened two index construction from the Hamiltonian cycles with a new weighted objective term.

The travelling salesman problem asks,

What is the shortest possible path around a list of cities that visits every city only once and returns to the starting city?

This can be reformulated as a Hamiltonian Cycle problem like above however now each edge (uv) carries an associated weight. The aim now becomes minimising this weighting when travelling across the graph $G(E, V)$ in a cycle. This can be neatly wrote as,

$$H_B = B \sum_{(uv) \in E} W_{uv} \sum_{j=1}^N x_{u,j} x_{v,j+1}$$

With two coefficients we must ensure that the hamiltonian cycle is never violated to prefer minimising weighting, this can be done with a choice of A larger than $B \times \max W_{uv}$

$$0 < B \max(W_{uv}) < A$$

So putting it all together we get our full travelling salesman energy function employing techniques(list a few)

$$\begin{aligned} H &= H_A + H_B = \\ &= A \sum_{v=1}^n \left(- \sum_i^N x_{v,j} + 2 \sum_{i < j}^N x_{v,i} x_{v,j} \right) \\ &+ A \sum_{j=1}^N \left(- \sum_{v=1}^n x_{v,j} + 2 \sum_{u < v}^n x_{u,j} x_{v,j} \right) \\ &+ A \sum_{(uv) \notin E} \sum_{j=1}^N x_{u,j} x_{v,j+1} \\ &+ B \sum_{(uv) \in E} W_{uv} \sum_{j=1}^N x_{u,j} x_{v,j+1} \end{aligned}$$

3.6.1 Implementing TSP

As the constraint term H_A is unchanged we will build it in the same way as the Hamiltonian cycle case above. The only difference will be in our construction of H_B , which will require attaching an edge weight to our graph.

Once again we will use networkx to implement this, using edge attributes to store these weight in our graph arrays. The simplest version assigns a random integer from $(1 \rightarrow 10)$ as the edge weight, will be taking every edge and assigning a random integer between 1 and 10

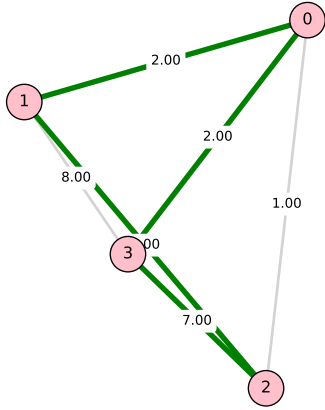
```
G = nx.complete_graph(4)
for u, v in G.edges():
    G[u][v]['weight'] = np.random.randint(1, 10)
```

When building Q , we read these weights back out of the graph array and add the $B W_{uv}$ contributions with a final loop.

```
weights = nx.get_edge_attributes(G, 'weight')
```

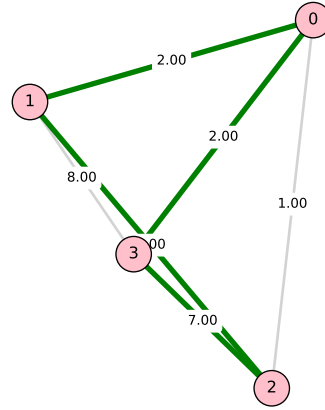
Validity checking is identical to the Hamiltonian cycle case above, only we want to add up the total weight of the cycle for comparison of these annealed solutions to brute force computations which can be seen below in Figure 3.5.

TSP Cycle |Type: Brute| Total weight = 12.00

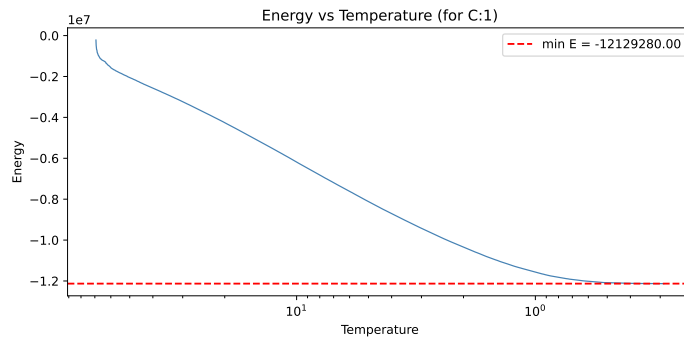


(a) Brute force random int TSP,
total weight = 13

TSP Cycle |Type: Annealing| Total weight = 12.00



(b) Annealed random int TSP,
total weight=13



(c) TvE for the annealed distance TSP
C refers to $A = C \cdot B \max(W_{uv})$

Figure 3.5: random in based weights $N = 4$

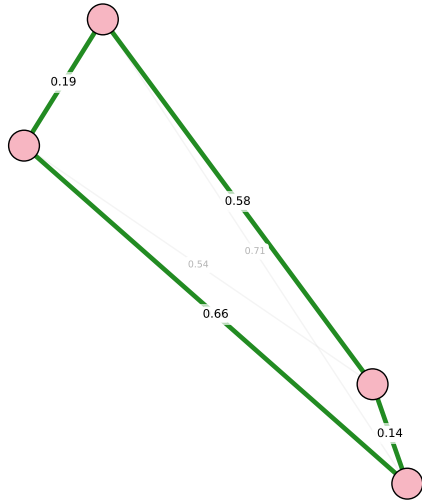
3.6.2 Distance Based Weighting

We can make TSP more realistic by computing edge weights from the distance between vertices, rather than assigning them randomly. Here we create a complete graph, randomly assign each vertex a position on the unit square and then calculate the distance associated with each edge.

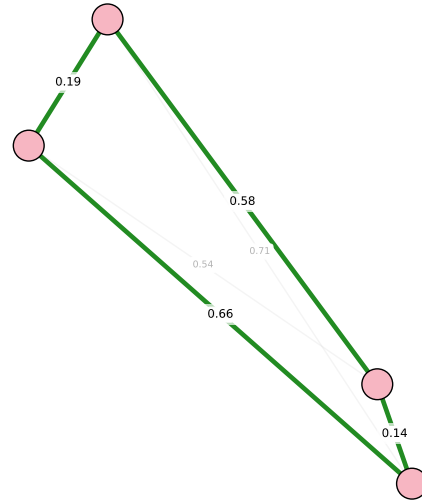
```
def distance_graph(N):
    pos = {i: (np.random.rand(), np.random.rand()) for i in range(N)}
    G = nx.complete_graph(N)
    for u, v in G.edges():
        dist = np.linalg.norm(np.array(pos[u]) - np.array(pos[v]))
        G[u][v]["weight"] = dist
    W = nx.to_numpy_array(G, weight="weight")
    return G, W, pos
```

For a small complete graph, we can compare the annealed solutions directly with the brute force results. Figure 3.6 one example of this where our brute force and annealed solutions agree both finding

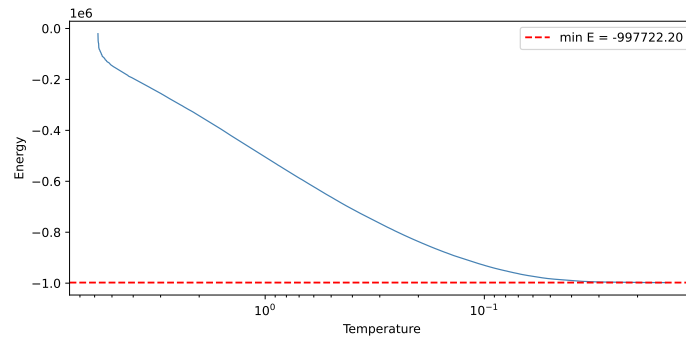
a solution with a total weight of 1.56, alongside a temperature versus energy plot of the annealing process itself.



(a) Brute force distance TSP,
total weight = 1.56



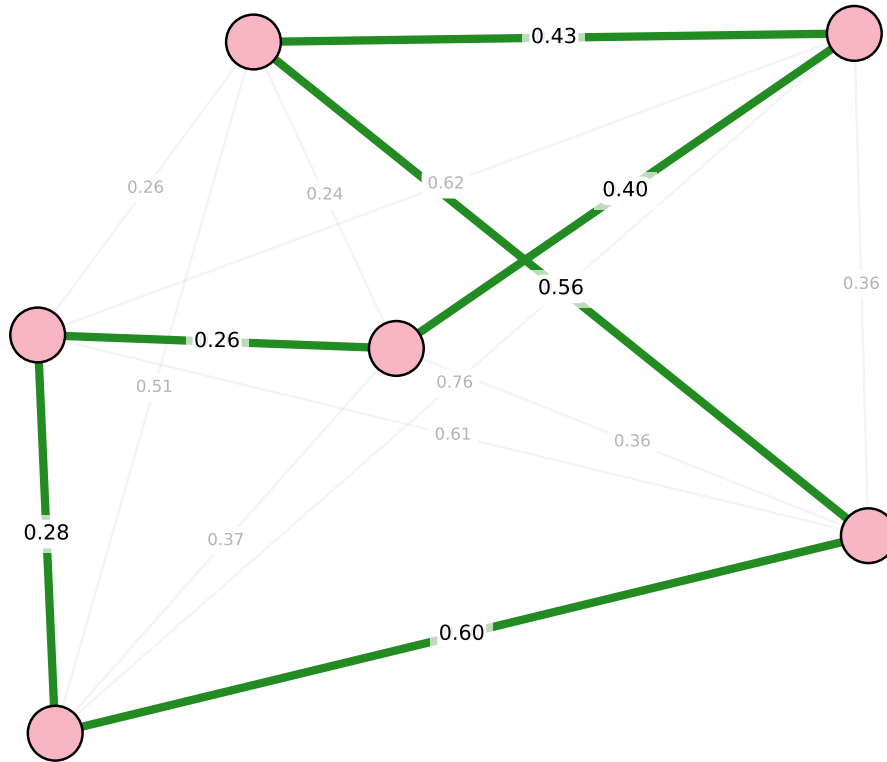
(b) Annealed distance TSP,
total weight=1.56



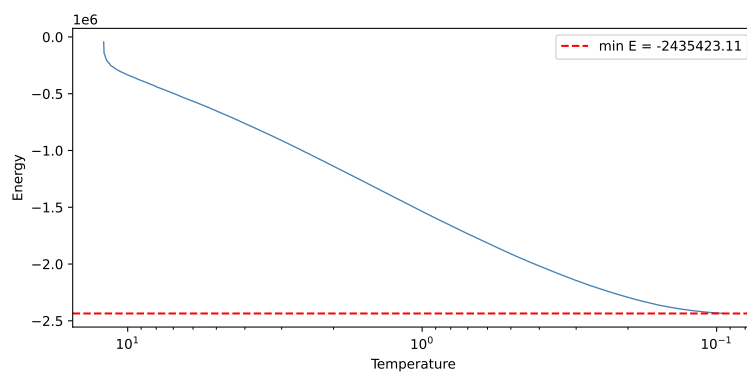
(c) TvE for the annealed distance TSP

Figure 3.6: distance based weights $N = 4$

Scaling this up to $N = 6$, Figure 3.7, brute force checking becomes considerably time consuming and the complete graph visualisation gets increasingly cluttered as more edges are required. Our annealing recovers a total weight of 2.53. This illustrates why annealing is a valuable tool as problems scale, exact methods become unattainable while annealing continues to recover results. We can scale N up and try annealing, however the plots do become increasing messy with the nature of a complete graph.



(a) Annealed distance TSP,
total weight=1.56



(b) TvE for the annealed distance TSP

Figure 3.7: distance based weights $N = 6$

3.7 Concluding the Combinatorial Problems

Across this sections, we've worked through a sequence of increasingly complex combinatorial problems: a single squared penalty term for number partitioning, introducing an objective term & balancing two components with A/B ratio in graph partition, multi index variables for graph colouring and position ordering constraints for Hamiltonian cycles, all building up to the travelling salesman problem with a weighted objective on top of a Hamiltonian cycle.

Every one of these problems was built the same way: express the constraints as binary conditions, then penalise deviations from them with squared terms and derive a constraint on A/B to ensure the ground state does not violate the problem's constraints.

4 QUBO Formulation of Rocket Optimal Control

The problems in the previous section (partitioning, graph colouring, Hamiltonian cycles, & TSP) all share one feature, every variable in the problem is naturally binary, however many real world optimisation problem are not naturally discrete at all. Rocket trajectory control is the example we will use: continuous quantities of velocity & position evolving smoothly overtime with respect to physics based dynamics and binary engine thrusts ⁴.

This raises the problem we didn't encounter in the previous section, how to encode a continuous quantity using only binary variable? In order to use the QUBO annealing framework we've built throughout this project we need work through the process of encoding continuos states into binary states.

4.1 Encoding Continuos Variables

In our rocket control problem, position x and velocity v are both continuos, this doesn't cleanly map onto our QUBO annealing framework since $E = \mathbf{x}^T Q \mathbf{x}$ requires $x_i \in \{0, 1\}$. Problems of this kind, combining continuos and discrete variables are known as mixed-integer problems. We followed the encoding techniques set out by A.Iftakher in *Solving mixed-integer problems as QUBO: Encodings, reformulations, and rolling-precision algorithm*. [6]

4.1.1 Integer Encoding

Take $x_t \in \mathbb{Z}$ with bounds $x_t \in \{L_t, L_t + 1, \dots, U_t\} \subset \mathbb{Z}$. We define the range $R_t = U_t - L_t$. With B binary variables we can represent 2^B distinct values, $\{0, 1, \dots, 2^B - 1\}$. To cover the full range we require $2^B - 1 \geq R_t$, or equivalently $B_t = \log_2(R_t + 1)$ binary variables.

Introducing binary variables $z_{t,0}, z_{t,1}, \dots, z_{t,2^{B_t}-1} = \{0, 1\}$, we can encode our x_t as,

$$x_t = L_t + \sum_{j=0}^{B_t-1} 2^j z_{t,j}$$

This works well for integers, but we need to take this further to continuos variables.

⁴Real rocket control doesn't have binary thrust but this is the simplest model

4.1.2 Decimal place expansion

Take a continuous variable bounded by $x_t^L \leq x_t \leq x_t^U$, and rescale it to $\tilde{x}_t \in [0, 1]$.

$$\tilde{x}_t = \frac{x_t - x_t^L}{x_t^U - x_t^L} \quad \text{where} \quad x_t = x_t^L + (x_t^U - x_t^L)\tilde{x}_t$$

Then we approximate $\tilde{x}_t$ with a finite decimal expansion to J digits, using ten binary variables, $z_{i,j,t}$. We use one $z_{i,j,t}$ per active digit, representing $0 \rightarrow 9$, and a single endpoint bit.

$$\tilde{x}_t = \underbrace{\sum_{j=1}^J \sum_{i=1}^{I=9} 10^{-j} z_{i,j,t}}_{J \text{ decimal places}} + 10^{-J} z_{t,m}$$

The endpoint bit is needed as $\sum_{j=1}^J 9 \cdot 10^{-j} = 1 - 10^{-J}$, without it the encoding could never reach $\tilde{x}_t = 1$. This is the simplest possible decimal encoding, but its expensive it requires $(9J + 1)$ binary variables per continuous variable scaling roughly 9 bits per decimal place.

4.1.3 Weighted Encoding (four bits per variable)

Rather than using nine separate bits per digit, we can employ a more efficient encoding method. We can represent $0 \rightarrow 9$ using only four binary variables

$$\begin{aligned} \tilde{x}_t &= \underbrace{\sum_{j=1}^J 10^{-j} d_{t,j}}_{J \text{ decimal places}} + \underbrace{10^{-J} z_{t,m}}_{\text{endpoint bit}} \\ &= \underbrace{\sum_{j=1}^J 10^{-j} (z_{t,j,1} + 2z_{t,j,2} + 3z_{t,j,3} + 3z_{t,j,4})}_{\text{weighted encoding with four bits per digit}} + \underbrace{10^{-J} z_{t,m}}_{\text{endpoint bit}} \end{aligned}$$

These weights are chosen so that all sixteen combinations of the four bits can reach every integer from $0 \rightarrow (1 + 2 + 3 + 3) = 9$, using only $(4J + 1)$ binary bits per continuous variable.

Both decimal encodings introduce a truncation error bounded by error $\leq 10^{-J}$, from representing continuous numbers as finite numbers of decimal places.

4.2 Rocket Take-off Problem Formulation

Having established how to encode continuous variable in binary, we can now build a concrete mixed-integer optimal control problem *a 1D rocket take off*. The goal throughout is to construct an energy function $E = \mathbf{x}^T Q \mathbf{x}$, with x binary, whose minimum corresponds to a valid optimal trajectory.

4.2.1 Physical Model

We consider a rocket moving in one dimension, with

$$\begin{aligned}x(t) &= \text{height} \\ \dot{x}(t) = v(t) &= \text{velocity} \\ u(t) &= \text{engine thrust.}\end{aligned}$$

We take the simplest possible thrust model, the engine is either fully on or fully off which is controlled with a binary control variable $b(t)$.

$$u(t) = T_{\max} b(t)$$

Applying Newtons second law,

$$\begin{aligned}F &= ma \\ F &= Tu_t - mg \\ a_t &= \frac{u}{m}u_t - g\end{aligned}$$

which gives us the system's equations of motion

$$\begin{cases} \dot{x} = v \\ \dot{v} = \frac{T_{\max}}{m}b(t) - g \end{cases}$$

4.2.2 Time Discretisation

For our computational purposes, we discretise time using the Euler method with some time step Δt , giving

$$\begin{cases} x_{k+1} = x_k + \Delta t v_k \\ v_{k+1} = v_k + \Delta t \left(\frac{T_{\max}}{m} b_k - g \right) \end{cases}$$

where $b_k \in \{0, 1\}$ is our binary control variable at time step k , and x_k, v_k are the continuous state variables which we will encode with the weighted four bit encoding from above.

4.2.3 Optimisation Problem

With the dynamics found, we can formulate the problem as a mixed-integer optimal control problem. We take our object to be getting from our starting position & velocity to our ending position & velocity whilst minimising the fuel used, which corresponds to firing the engine for the least amount of time steps possible.

$$\begin{aligned}
& \min \sum_{k=0}^{N-1} b_k \\
& \text{s.t. } x_{k+1} = x_k + \Delta t v_k \\
& \quad v_{k+1} = v_k + \Delta t \left(\frac{T_{\max}}{m} b_k - g \right) \\
& \quad x_0 = 0, \quad v_0 = 0 \\
& \quad x_N = 10, \quad v_N = 0 \\
& \quad b_k \in \{0, 1\}
\end{aligned}$$

4.3 Constructing the Energy Function

With our problem stated in a constrained formulation we can convert this into an unconstrained QUBO formulation by building an energy function following the same principles we used throughout the previous section.

4.3.1 Fuel Objective

The objective we chose for this problem was minimising fuel, hence minimising the number of time steps the engine fires. This translates to a linear term in b_k our binary control variable, this acts as our H_B objective term.

$$E_{\text{fuel}} = \alpha \sum_{k=0}^{N-1} b_k$$

We will use α to tune the relative weighting of the fuel efficiency objective against our constraints, this mirrors the tuning between A and B we saw prior however it will become a challenge to find an analytic condition like we saw in Lucas' paper above.

4.3.2 Starting and Ending Constraints

Fixing the rockets boundary conditions of starting on the ground $x_0 = 0$ with no initial velocity $v_0 = 0$ and ending at $x_N = 10$ with also no velocity $v_N = 0$. We will penalise it with the same squared penalty method we've used throughout.

$$E_{\text{boundary}} = \sigma \left((x_N - x^*)^2 + (v_N - v^*)^2 \right) \quad \text{where } x^* \text{ \& } v^* \text{ are the targets}$$

σ will be used to tune the relative weighting of our boundary conditions.

4.3.3 Dynamics Constraints

From the two equations of motion we will construct the last two constraints, for position and velocity ensuring that our energy function will give us solutions that don't violate the physics of the situation.

Since the equations of motion hold at every time step we penalise any deviations away from these in the same squared manner,

$$\begin{aligned} E_{\text{dynamics}} &= \beta(E_{\text{x dynamics}}) + \gamma(E_{\text{v dynamics}}) \\ &= \beta \sum_{k=0}^{N-1} (x_{k+1} - x_k - \Delta t v_k)^2 + \gamma \sum_{k=0}^{N-1} \left(v_{k+1} - v_k - \Delta t \left(\frac{T_{\text{max}}}{m} b_k - g \right) \right)^2 \end{aligned}$$

Each term will be exactly zero when the dynamics are satisfied at each time step k and thus physically valid solutions. Here β and γ let us weight the position and velocity consistency independently, ensuring that no term dominates the energy function.

4.3.4 Complete Energy function

We have all the components required to build our rocket control energy function. H_A will consist of the dynamics terms and the boundary conditions, and H_B will be our objective fuel term.

$$\begin{aligned} H &= H_A + H_B \\ &= E_{\text{fuel}} + E_{\text{dynamics}} + E_{\text{BC}} \end{aligned}$$

Writing out each term in full,

$$E_{\text{fuel}} = \alpha \sum_{k=0}^{N-1} b_k$$

$$\begin{aligned} E_{\text{dynamics}} &= \beta(E_{\text{x dynamics}}) + \gamma(E_{\text{v dynamics}}) \\ &= \beta \sum_{k=0}^{N-1} (x_{k+1} - x_k - \Delta t v_k)^2 + \gamma \sum_{k=0}^{N-1} \left(v_{k+1} - v_k - \Delta t \left(\frac{T_{\text{max}}}{m} b_k - g \right) \right)^2 \end{aligned}$$

$$\begin{aligned} E_{\text{BC}} &= \sigma(E_{\text{Start}}) + \sigma(E_{\text{End}}) \\ &= \sigma \left((x_0 - x^{\star \text{ start}})^2 + (v_0 - v^{\star \text{ start}})^2 + (x_N - x^{\star \text{ end}})^2 + (v_N - v^{\star \text{ end}})^2 \right) \\ &\quad \text{where } x^{\star \text{ start/end}} \ \& \ v^{\star \text{ start/end}} \text{ are the targets} \end{aligned}$$

4.4 Building the QUBO Matrix

4.4.1 General Quadratic expansion

Before working through each term in the energy function individually, it's worth going through deriving the general form expansion of squared penalty terms. This is the same squared penalty terms we saw repeatedly in the last section, however once we go through the general approach we can apply it

to our fuel, dynamics and boundary condition terms.

Many of the penalty terms considered in this expansion are written in the form

$$E = \lambda \left(C + \sum_i f_i z_i \right)^2,$$

where C is a constant, λ is our constraint weighting, f_i are scalar coefficients, and $z_i \in \{0, 1\}$ are binary variables.

Expanding the square gives

$$E = \lambda \left(C^2 + 2C \sum_i f_i z_i + \sum_i \sum_j f_i f_j z_i z_j \right).$$

Since the variables are binary, $z_i^2 = z_i$, so

$$E = \lambda C^2 + \lambda \sum_i (f_i^2 + 2C f_i) z_i + 2\lambda \sum_{i < j} f_i f_j z_i z_j.$$

Therefore, the corresponding QUBO contributions are

$$\begin{aligned} Q_{ii} &+= \lambda (f_i^2 + 2C f_i), \\ Q_{ij} &+= 2\lambda f_i f_j, \quad i < j, \end{aligned}$$

while the constant contribution λC^2 are dropped as it doesn't affect the minimum. With this result we can apply this to each of our H_A terms as they follow this exact form, we just need to identity each term.

4.4.2 Fuel Contribution

Before we go about applying our general squared penalty result above, we can construct the simplest term in the QUBO representation the linear objective term with no squaring required.

As our fuel component E_{fuel} is linear it will be diagonal down b_k , with no off diagonal contributions since there is no quadratic term in our control variables b_k .

$$E_{\text{fuel}} = \alpha \sum_{k=0}^{N-1} b_k \implies Q_{b_k, b_k} = +\alpha$$

4.4.3 continuous encoding

Unlike with our fuel component, E_{Dynamics} and E_{BC} both contain continuous state variables of x_k and v_k . Take E_{Dynamics} ,

$$\beta \left(\sum_{k=0}^{N-1} \left(\underbrace{x_{k+1}}_{\text{not binary}} - \underbrace{x_k}_{\text{not binary}} - \Delta t \underbrace{v_k}_{\text{not binary}} \right) \right)^2 \cdots + \gamma \left(\sum_{k=0}^{N-1} \left(\underbrace{v_{k+1}}_{\text{not binary}} - \underbrace{v_k}_{\text{not binary}} - \Delta t \left(\frac{T_{\text{max}}}{m} b_k - g \right) \right) \right)^2$$

We use the four bit weighted decimal encoding introduced at the start of this section for both position and velocity state variables,

$$x_m = x_m^{\min} + (x_m^{\max} - x_m^{\min}) \left(\sum_{j=1}^J 10^{-j} (x_{t,j,1} + 2x_{t,j,2} + 3x_{t,j,3} + 3x_{t,j,4}) + 10^{-J} x_{t,m} \right)$$

$$v_m = v_m^{\min} + (v_m^{\max} - v_m^{\min}) \left(\sum_{j=1}^J 10^{-j} (v_{t,j,1} + 2v_{t,j,2} + 3v_{t,j,3} + 3v_{t,j,4}) + 10^{-J} v_{t,m} \right)$$

To keep manageable notation we define each continuous variable as a weighted sum of binary bits,

$$x_k = A^x + w^x \cdot z_k^x \quad v_k = A^v + w^v \cdot z_k^v$$

where our offset is defined $A^x = x_k^{\min}$ and $A^v = v_k^{\min}$. We now have two sets of binary bits, z_k^x and z_k^v , encoding our continuous states with our weighting defined like below,

Variable	Weight w_i^x
$z_{k,1,1}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-1} \cdot 1$
$z_{k,1,2}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-1} \cdot 2$
$z_{k,1,3}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-1} \cdot 3$
$z_{k,1,4}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-1} \cdot 3$
$z_{k,2,1}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-2} \cdot 1$
$z_{k,2,2}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-2} \cdot 2$
$z_{k,2,3}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-2} \cdot 3$
$z_{k,2,4}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-2} \cdot 3$
$\vdots$	$\vdots$
$z_{k,end}^x$	$(x^{\max} - x^{\min}) \cdot 10^{-J}$

with the equivalent table for w^v .

With this encoding we can substitute x_k & v_k into our energy terms and find a constant and linear in z term, which we know to expand as it matches our general squared penalty term expansion.

The full binary vector collects every control and state variable at each time step,

$$\mathbf{z} = (b_0, \dots, b_{N-1}, \mathbf{x}_0, \dots, \mathbf{x}_N, \mathbf{v}_0, \dots, \mathbf{v}_N)^T$$

where each x_k and v_k is expanded to a full binary variable as,

$$\mathbf{x}_k = (x_{k,m}, x_{k,1,1}, x_{k,1,2}, x_{k,1,3}, x_{k,1,4}, \dots, x_{k,J,1}, x_{k,J,2}, x_{k,J,3}, x_{k,J,4})$$

$$\mathbf{v}_k = (v_{k,1,1}, v_{k,1,2}, v_{k,1,3}, v_{k,1,4}, \dots, v_{k,J,1}, v_{k,J,2}, v_{k,J,3}, v_{k,J,4})$$

This gives us our binary variable applicable for our QUBO representation of $E = \mathbf{z}^T Q \mathbf{z}$.

4.4.4 Terminal and starting contributions

To give us our boundary conditions for our rocket take off problem, the rocket starts at rest on the ground and needs to reach the target height with zero velocity, $x_0^* = 0$, $v_0^* = 0$, $x_N^* = 10$, $v_N^* = 0$. We will enforce this with the standard squared constraints penalising deviations from the targets. The full boundary condition term,

$$\begin{aligned} E_{\text{BC}} &= \sigma(E_{\text{Start}}) + \sigma(E_{\text{End}}) \\ &= \sigma((x_0 - x^{\star \text{ start}})^2 + (v_0 - v^{\star \text{ start}})^2 + (x_N - x^{\star \text{ end}})^2 + (v_N - v^{\star \text{ end}})^2) \end{aligned}$$

Each of the four terms will have the same structure, so we'll work through one $(x_0 - x^{\star \text{ start}})^2$ in detail. Substituting in the binary encoding $x_0 = A^x + \mathbf{w}^x \cdot \mathbf{z}_0^x$

$$\begin{aligned} \sigma(x_0 - x_0^*)^2 &= \sigma(C_0^x + w^x \cdot z_0^x)^2 \\ &= \sigma(C_0^x)^2 + \sigma(w^x \cdot z_0^x)^2 + 2\sigma C_0^x w^x \cdot z_0^x \end{aligned}$$

which matches our general form of $\lambda(C + \sum_i f_i z_i)^2$ with $\lambda = \sigma$, $C = C_0^x$, $f_i = w_i^x$.

$$\begin{aligned} Q_{ii} &+= \sigma((w_i^x)^2 + 2C_0^x w_i^x) \\ Q_{ij} &+= 2\sigma w_i^x w_j^x, \quad i < j \end{aligned}$$

for the bits of z_0^x . The remaining three terms $(v_0 - v_0^*)^2$, $(x_N - x_N^*)^2$, $(v_N - v_N^*)^2$ each contribute their equivalent term to Q , in the same manner mainly with a different constant,

$$C_0^v = A^v - v_0^*, \quad C_N^x = A^x - x_N^*, \quad C_N^v = A^v - v_N^*$$

and applied over their respective bits of z .

4.4.5 Dynamic Constrains

The last contribution to Q comes from the two dynamics constraints on position and velocity at every time step.

$$\begin{aligned} E_{\text{dynamics}} &= \beta(E_{\text{x dynamics}}) + \gamma(E_{\text{v dynamics}}) \\ &= \beta \sum_{k=0}^{N-1} (x_{k+1} - x_k - \Delta t v_k)^2 + \gamma \sum_{k=0}^{N-1} \left(v_{k+1} - v_k - \Delta t \left(\frac{T_{\text{max}}}{m} b_k - g \right) \right)^2 \end{aligned}$$

Unlike with our boundary conditions, we mix different time steps in our sums and our control variable b_k so we now need to be careful with our indexing when we collect our terms rather than them being in one block like last time.

We'll look at $\beta(E_{\text{x dynamics}})$ first,

$$\begin{aligned}
E_{\text{x dynamics}} &= \beta \sum_{k=0}^{N-1} (x_{k+1} - x_k - \Delta t v_k)^2 \\
&= \beta \sum_{k=0}^{N-1} \left(A^x + \mathbf{w}^x \cdot \mathbf{z}_{k+1}^x - (A^x + \mathbf{w}^x \cdot \mathbf{z}_k^x) - \Delta t (A^v + \mathbf{w}^v \cdot \mathbf{z}_k^v) \right)^2 \\
&= \beta \sum_{k=0}^{N-1} \left(\underbrace{-\Delta t A^v}_{\text{const}} + \underbrace{\sum_i w_i^x z_{k+1,i}^x - \sum_i w_i^x z_{k,i}^x - \Delta t \sum_i w_i^v z_{k,i}^v}_{\text{linear in } \mathbf{z}} \right)^2
\end{aligned}$$

This again is in the general form $\lambda(C + \sum_i f_i z_i)^2$, this time with $C_k^x = -\Delta t A^v$ and coefficients $f_{k,i}^x$ defined,

$$f_{k,i}^x = \begin{cases} +w_i^x & z_i \in \mathbf{z}_{k+1}^x \\ -w_i^x & z_i \in \mathbf{z}_k^x \\ -\Delta t w_i^v & z_i \in \mathbf{z}_k^v \\ 0 & \text{otherwise} \end{cases}$$

This gives us Q contributions of

$$\begin{aligned}
Q_{ii} &+= \beta (f_{k,i}^x)^2 + 2\beta C_k^x f_{k,i}^x, & \text{where } C_k^x &= -\Delta t A^v \\
Q_{ij} &+= 2\beta f_{k,i}^x f_{k,j}^x \quad i < j
\end{aligned}$$

summing over $k = 0, \dots, N-1$, with the sum mixing bit blocks summed over z_{k+1}^x, z_k^x, z_k^v at each step.

For the velocity we apply the same approach, now for

$$\begin{aligned}
E_{\text{v dynamics}} &= \gamma \sum_{k=0}^{N-1} \left(v_{k+1} - v_k - \Delta t \left(\frac{T_{\max}}{m} b_k - g \right) \right)^2 \\
&= \gamma \sum_{k=0}^{N-1} \left(\underbrace{\Delta t g}_{\text{const}} + \underbrace{\sum_i w_i^v z_{k+1,i}^v - \sum_i w_i^v z_{k,i}^v - \Delta t \frac{T_{\max}}{m} b_k}_{\text{linear in } \mathbf{z}} \right)^2
\end{aligned}$$

with a constant $C_k^v = \Delta t g$ and coefficients

$$f_{k,i}^v = \begin{cases} +w_i^v & z_i \in \mathbf{z}_{k+1}^v \\ -w_i^v & z_i \in \mathbf{z}_k^v \\ -\Delta t \frac{T_{\max}}{m} & z_i = b_k \\ 0 & \text{otherwise} \end{cases}$$

Giving us our Q contributions of

$$\begin{aligned} Q_{ii} &+= \gamma (f_{k,i}^v)^2 + 2\gamma \Delta t g f_{k,i}^v, \text{ where } C_k^v = \Delta t g \\ Q_{ij} &+= 2\gamma f_{k,i}^v f_{k,j}^v \quad i < j \end{aligned}$$

Together, both position and velocity dynamics constraint terms couple consecutive pairs of time steps in our QUBO representation. This is unlike the fuel and boundary terms prior which only added to their own respective blocks.

4.5 Final Q Matrix

Finally we can collect the contributions from every component above, we

$$\begin{aligned} E_{\text{fuel}} &\implies Q_{b_k, b_k} += \alpha \\ E_{\text{BC}} &\implies \begin{cases} Q_{ii} += \sigma (w_i^2 + 2C w_i) \\ Q_{ij} += 2\sigma w_i w_j, \quad i < j \end{cases} \quad \text{for each of } C \in \{C_0^x, C_0^v, C_N^x, C_N^v\} \\ E_{\text{x dynamics}} &\implies \begin{cases} Q_{ii} += \beta (f_{k,i}^x)^2 + 2\beta C_k^x f_{k,i}^x \\ Q_{ij} += 2\beta f_{k,i}^x f_{k,j}^x, \quad i < j \end{cases} \\ E_{\text{v dynamics}} &\implies \begin{cases} Q_{ii} += \gamma (f_{k,i}^v)^2 + 2\gamma C_k^v f_{k,i}^v \\ Q_{ij} += 2\gamma f_{k,i}^v f_{k,j}^v, \quad i < j \end{cases} \end{aligned}$$

Where, $C_k^v = \Delta t g$ and $C_k^x = -\Delta t A^v$ for the dynamics.

With this expression giving us our Q matrix of our rocket take off problem, we can now go about assembling and annealing this problem.

4.6 Implementation

With our rocket problem formulated as a QUBO Q matrix we can apply the same used throughout the previous section: build Q , anneal, decode and check validity.

4.6.1 Indexing and Encoding

The first main challenge with this problem compared to previous problems we've dealt with is indexing. Similar our index flattening of the prior Hamiltonian cycles flattening $(v, i) \rightarrow k = v \cdot n + i$ into one index we can work with, however now we need to map (control, position, velocity) to one index with each varying in size. We can handle this with a single indexing function, that can index our point based on which variable type it is and time step,

```
def idx(var, k, bit, N, J):
    B = 4 * J + 1
    if var == "b":
        return k
    elif var == "x":
```

```

        return N + k * B + bit
    elif var == "v":
        return N + (N + 1) * B + k * B + bit
    else:
        raise ValueError(f"Unknown variable type: {var!r}")

```

This lays our N control bits b_k first, then all the position state bits $4J + 1$ bits per time step and lastly $4J + 1$ velocity bits per time step.

Now the most different part of this implementation is that of the encoding of continuous state variables into binary. We can build the weight vector w and offset A , for each variable directly with a function,

```

def encoding_coeffs(lower, upper, J):
    span = upper - lower
    multipliers = [1, 2, 3, 3]
    weights = [span * 10 ** (-J)]
    for j in range(1, J + 1):
        scale = 10 ** (-j)
        weights.extend(span * scale * m for m in multipliers)
    return weights, lower

```

4.6.2 The General Expansion

The general squared penalty term expansion as we derived above, can be built as a single reusable function with f_i being imputed as a *terms* dictionary.

```

def add_square(Q, terms, C, lam):
    items = list(terms.items())

    for i, fi in items:
        Q[i, i] += lam * (fi ** 2 + 2 * C * fi)

    for (i, fi), (j, fj) in combinations(items, 2):
        lo, hi = (i, j) if i < j else (j, i) ## upper triangle
        Q[lo, hi] += 2 * lam * fi * fj

```

This function makes building the rest of our constraints much more efficient which is crucial as the amount of terms increases.

4.6.3 Building Each Energy Term

With `idx`, `encoding_coeffs`, and `add_square` we can go about constructing each energy term separately as its own short function which we can then call for a final build `Q` function.

The fuel is the simplest, as it is linear and not in the squared penalty form,

```
def add_fuel(Q, N, J, a):
    for k in range(N):
        i = idx("b", k, None, N, J)
        Q[i, i] += a
```

The boundary cost function can be used to handle both the starting and initial energies, when ran for $k = 0$ and $k = N$. We build each variables terms dictionary from its encoding weights and passing those directly into `add_square`

```
def add_boundary_cost(Q, N, J, weight, k, xmin, xmax, vmin, vmax,
                     x_target=0.0, v_target=0.0):
    wx, Ax = encoding_coeffs(xmin, xmax, J)
    terms = {idx("x", k, bit, N, J): w for bit, w in enumerate(wx)}
    add_square(Q, terms, Ax - x_target, weight)

    wv, Av = encoding_coeffs(vmin, vmax, J)
    terms = {idx("v", k, bit, N, J): w for bit, w in enumerate(wv)}
    add_square(Q, terms, Av - v_target, weight)
```

The dynamics terms implementation are the most difficult due to the coupling of time steps. For the position dynamics we build a dictionary spanning z_k^x, z_{k+1}^x and z_k^v , we then run three loops building our dictionary covering each of the three cases for $f_{k,i}^x$,

$$f_{k,i}^x = \begin{cases} +w_i^x & z_i \in \mathbf{z}_{k+1}^x \\ -w_i^x & z_i \in \mathbf{z}_k^x \\ -\Delta t w_i^v & z_i \in \mathbf{z}_k^v \\ 0 & \text{otherwise} \end{cases}$$

With this dictionary built we can run it through our `add_square` once for each time step,

```
def add_position_dynamics(Q, N, J, beta, dt, xmin, xmax, vmin, vmax):
    wx, Ax = encoding_coeffs(xmin, xmax, J)
    wv, Av = encoding_coeffs(vmin, vmax, J)
    for k in range(N):
        terms = {}
        for bit, w in enumerate(wx):
            gi = idx("x", k + 1, bit, N, J)
            terms[gi] = terms.get(gi, 0.0) + w
        for bit, w in enumerate(wx):
            gi = idx("x", k, bit, N, J)
            terms[gi] = terms.get(gi, 0.0) - w
        for bit, w in enumerate(wv):
            gi = idx("v", k, bit, N, J)
            terms[gi] = terms.get(gi, 0.0) - dt * w
        C = -dt * Av
        add_square(Q, terms, C, beta)
```

Velocity dynamics follows the exact same structure with a differently defined $f_{k,i}^x$, and hence a different loop

```
...
for k in range(N):
    terms = {}
    for bit, w in enumerate(wv):
        g_idx = idx("v", k + 1, bit, N, J)
        terms[g_idx] = terms.get(g_idx, 0.0) + w
    for bit, w in enumerate(wv):
        g_idx = idx("v", k, bit, N, J)
        terms[g_idx] = terms.get(g_idx, 0.0) - w

    b_i = idx("b", k, None, N, J)
    terms[b_i] = terms.get(b_i, 0.0) - dt * Tmax / mass
...
```

4.6.4 Assembling Q

Finally each of these functions are called building all the different components contributions into a shared Q , since our construction of `add_square` only adds to the upper triangle we then symmetrise Q at the end to match the expectations of our annealing framework.

```
...
Q = (Q + Q.T)/2.0
return Q
```

4.6.5 Decoding a solution

After annealing with our Q and producing a lowest energy state, we reverse the binary encoding summing over the bits and their respective weights,

```
...
b = [z[idx("b", k, None, N, J)] for k in range(N)]

x = []
for k in range(N + 1):
    bits = [z[idx("x", k, bit, N, J)] for bit in range(len(wx))]
    x.append(Ax + sum(w * bi for w, bi in zip(wx, bits)))

v = []
for k in range(N + 1):
    bits = [z[idx("v", k, bit, N, J)] for bit in range(len(wv))]
    v.append(Av + sum(w * bi for w, bi in zip(wv, bits)))
...
```

With Q assembled and a decoding function in place we can go about annealing the rocket problem and verifying if this construction produces realistic trajectories.

4.7 Annealing and Results

With our Q assembled we can use the same annealing framework built up in section 2/1. However, this surfaced issues with our original annealing implementation that only became apparent once applied to a rocket problem of this scale.

4.7.1 Not finding the Minima

Our first runs consistently returned nonsense plots where there was no correlation between position, velocity or thrust and no regard for objectives. The final energies were on the order 10^8 regardless of varying parameters such as *sweeps_per_T*. This lead us to suspecting our annealer itself wasn't finding anywhere close to the true minima, rather than an issue with the formulation of Q . To diagnose our issue we benchmarked against D-wave's simulated annealing package, *neal*.

```
sampler =neal.SimulatedAnnealingSampler()  
sampleset = sampler.sample_qubo(Q_dict, num_sweeps=5000)
```

D-wave's sampler returned minima of the order -10^5 , well below what we were reaching with our annealer confirming an issue in the annealer rather than the problem itself being poorly tuned with coefficients.

Interestingly D-wave's *neal* requires almost no tuning compared to ours, notably no T_{Start} or T_{End} just a sweep count. This inspired us to implement a T_{Start} and T_{End} calculator based off hot and cold probabilities of acceptance for each problems energy scale.

Our problem ended up being the scale of our coefficients $\alpha, \beta, \gamma, \sigma$. As we chose larger values in order to ensure constraints were followed, we directly increased the scale of $|dE|$ for a single flip and since

$$T \propto \frac{-dE}{\ln p}$$

With our original fixed starting value of $T_{\text{Start}} \approx 50$ chosen without reference to the problems energy scale, (only chosen as it worked prior for all the combinatorial problem we saw in section 3), we were essentially starting with a cold acceptance rate rather than a hot one. The annealer in practice was rarely accepting any flips from the start and unable to explore the landscape at all, explaining why increasing sweeps per temperature wasn't fixing the problem.

Once we improved our annealer to calculate T_{Start} and T_{End} for the specific problem at hand based off imputed cold and hot acceptance rates instead, we resolved the issue. We then recovered energies matching D-wave.

4.8 Tuning the constraint coefficients

Now with our annealer correctly finding the true minima of the QUBO, the remaining problem is whether the minima found corresponds to realistic, physically valid solutions. Previously, when we faced this issue in the combinatorial problems in section 2 we had hard analytical constraint, such as for the travelling salesman problem where we ensured that the Hamiltonian cycle was never violated in favour of minimising travelling distance by setting our coefficients such that

$$0 < B \max(W_{uv}) < A$$

Here however we have 4 coefficients $\alpha, \beta, \gamma, \sigma$ rather than a single clean ratio of A & B , and the coupling between position and velocity dynamics makes finding an analyticity solution considerably harder. Its reasonable that a constraint does exist but we didn't look for it, instead we tuned these coefficients numerically with dynamics residuals as our diagnostic.

4.8.1 Residuals

When we look at our constraints in the general form

$$\sum_i f_i z_i = C \quad (4.1)$$

our residual of the constraint tells us how far its deviates this constraint. For the general form we define the residual,

$$r_{\text{general}} = \sum_i f_i z_i - C \quad (4.2)$$

Now for the position and velocity residuals they are,

$$\begin{aligned} r_{x_k} &= x_{k+1} - x_k - \Delta t v_k \\ r_{v_k} &= v_{k+1} - v_k - \Delta t \left(\frac{T_{\text{max}}}{m} b_k - g \right) \end{aligned}$$

Both of these residuals should be zero for any physically valid trajectory, this is what E_{dynamics} is penalising. However since the final QUBO energy is a sum of all constraints and objectives, without proper tuning the QUBO can favour other constraints over others due to scales this is why its useful to check individual residuals. This can be implemented into python with a couple loops

```
...
for k in range(N):
    pos_resid = x[k+1] - x[k] - dt * v[k]
    pos_residuals.append(pos_resid)

    vel_resid = v[k+1] - v[k] - dt * (Tmax/mass * b[k] - g_grav)
    vel_residuals.append(vel_resid)
...
```

With this diagnostic in hand, we can iteratively tune β and γ by annealing, decoding, checking residuals and adjusting appropriately. In the end we found a ratio of

$$\frac{\beta}{\gamma} \approx 100$$

between our two dynamics constraints gave physically realistic results with final coefficient choices of

```

a=2,
beta=10000*100,
gamma=10000,
sigma=10000,

```

We recover a plausible result in Figure 4.1, where $x_0 = 0$, $v_0 = 0$ and $x_N = 5$, however $v_N \approx 9$. The velocity not quite converging to the desired $v_N = 10$ reflects the nature of this penalty based constraint method: the annealer trades off a small residual violation on v_N for a lower total energy elsewhere in the trajectory, as we've emphasised not breaking the physical dynamics. With more time tweaking our coefficients numerically, or even finding an analytical solution, we're confident we could get this much closer to the exact result.

The rest of this result is plausible yet not especially exciting, turning the engine on and off is the most efficient strategy. This is still quite a small lattice size, but the number time steps scales this problem up drastically. Nonetheless, this works as a proof of concept, we are able to transform mixed-integer problems of this form into a QUBO solvable by an Ising solver, and hence in a form more natural for a probabilistic computer.

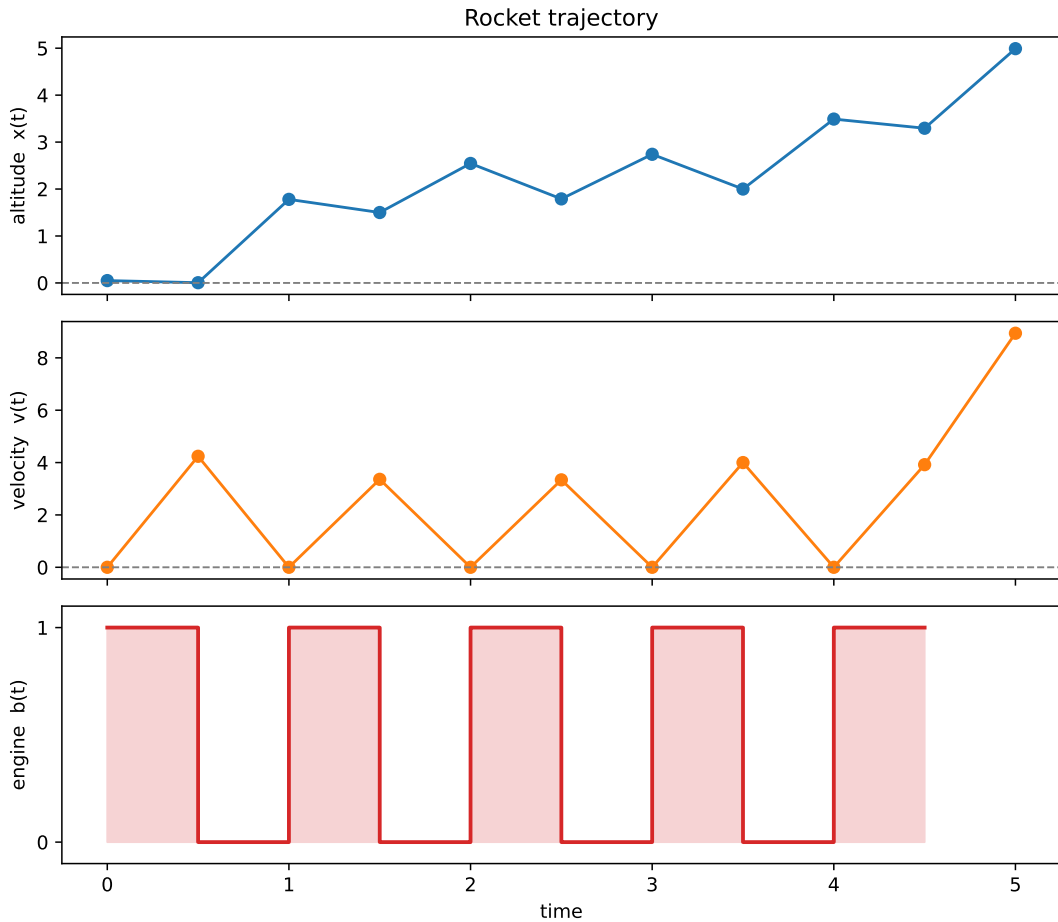


Figure 4.1: Our Rocket trajectory

$x_0 = 0$, $v_0 = 0$ and $x_N = 5$ $v_N = 10$

with time steps amount $N = 10$, time step size $\Delta t = 0.5$ and $J = 3$ decimal places in our expansion.

5 Conclusion

In this project, we reformulated a variety of optimisation problems as searches for the minimum of energy landscapes, using Ising and QUBO formulations. Starting from the Metropolis algorithm we built a framework of simulated annealing which allowed us to sample a system to recover the lowest energy state efficiently.

Applying this framework to a collection of established combinatorial problems, *number partitioning*, *graph partitioning*, *graph colouring*, *Hamiltonian cycles*, and *the Travelling Salesman Problem*, allowed us to building up a method for converting problems into penalty based constraint terms H_A and objective terms H_B . This included the importance of the ratio of coefficients A/B ensuring the ground state still satisfies both the constraints, while maximising objectives.

We extended this from solely discrete combinatorial problems to mixed-integer problems with our reformulation of 1D rocket trajectory control. By encoding our continuous parameters through binary decimal expansion, and enforcing rocket dynamics through squared penalty terms like as seen in the previous section, we were able to express this type of problem in a QUBO formulation. Annealing this with our established framework gave plausible results, correctly satisfying the dynamics and initial conditions, and reaching close to the target of $v_N = 10$, with $v_N \approx 9$. This reflects a shortcoming of this method as numerically tuning the coefficients as we did can lead to the annealer to trade off a small residual violation on v_N for a lower total energy elsewhere in the trajectory.

This project shows that problems such as 1D rocket trajectory optimisation can be reformulated into energy minimisation problems solvable through MCMC annealing methodology, and hence suited to probabilistic hardware.

References

- [1] Kerson Huang. *Statistical mechanics*. John Wiley & Sons, 1987.
- [2] Mark Newman and Gerard Barkema. “Monte carlo methods in statistical physics chapter 1-4”. In: *New York, USA* (1999).
- [3] Andrew Lucas. “Ising formulations of many NP problems”. In: *arXiv preprint arXiv:1302.5843* (2013).
- [4] Kenneth Appel and Wolfgang Haken. “Every planar map is four colorable”. In: (1976).
- [5] Twotwos. *File: Bridges of Königsberg.png*. https://commons.wikimedia.org/wiki/File:Bridges_of_Königsberg.png. Wikimedia Commons. Licensed under Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0). Accessed 15 July 2025. n.d.
- [6] Ashfaq Iftakher, Metin Türkay, and MM Faruque Hasan. “Solving mixed-integer problems as QUBO: Encodings, reformulations, and rolling-precision algorithm”. In: *Computers & Chemical Engineering* (2026), p. 109757.